

Graph Mining and Graph Representation Learning

Prof. O-Joun Lee

Dept. of Artificial Intelligence,
The Catholic University of Korea
ojlee@catholic.ac.kr

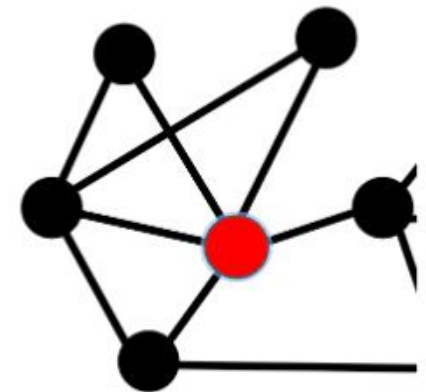
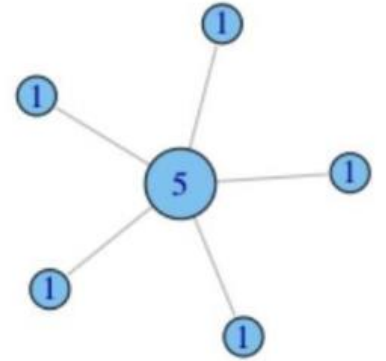
Contents



- Centrality Measurements:
 - Betweenness, Closeness, PageRank Centrality
- Graph Measurements:
 - Clustering Coefficients, Conductance, Modularity
- Additional Metrics:
 - Jaccard's Coefficient, Katz Index
- Graph Kernels:
 - Shortest Path Kernel, Weisfeiler-Lehman Kernel
- Graph Representation Learning (GRL):
 - From traditional ML for graph to GRL
 - WalkLet, Node2Vec, LINE, HOPE, Struc2Vec, Role2Vec, metapath2vec, HIN2Vec, JUST
 - TIMERS, DynamicTriad, Dynnode2vec, Continuous-Time Dynamic Network Embeddings

- Graphs efficiently model relationships, perfect for addressing questions like "What's the lowest-cost path from A to B?"
- We need a data structure that represents graphs
- Determining the "Best" Data Structure can depend on:
 - Properties of the graph (dense vs. sparse)
 - Common queries
 - For example: "is (u, v) an edge?" vs "what are the neighbors of node u ?"
- There are two standard graph representations:
 - Adjacency Matrix
 - Adjacency List

- **Node degree:** the degree k_v of node v is the number of edges (neighboring nodes) the node has
 - In-degree: Number of incoming edges to the node
 - Out-degree: Number of outgoing edges from the node
 - Total degree: Sum of in-degree and out-degree
- Knowing the network structure, we can calculate various useful quantities or measures that capture important features of network topology
- Centrality measures represent the most important nodes in graphs:
 - The most influential person in a social network
 - The most critical nodes in an infrastructure
 - The highest spreaders of disease



➤ Betweenness Centrality:

- A node is important if it **lies on many shortest paths** between other nodes

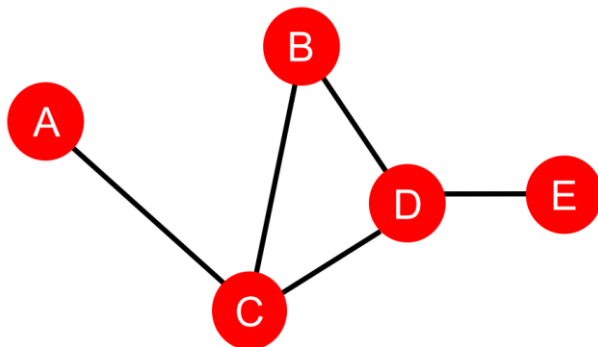
$$c_v = \sum_{s \neq v \neq t} \frac{\#(\text{shortest paths between } s \text{ and } t \text{ that contain } v)}{\#(\text{shortest paths between } s \text{ and } t)}$$

- Usually normalized by:

$$\bar{B}(v_i) = B(v_i) / [(n-1)(n-2) / 2]$$

No. of nodes in the graph

- For example:



- $c_A = c_B = c_E = 0$
- $c_C = 3$ (A-C-B, A-C-D, A-C-D-E)
- $c_D = 3$ (A-C-D-E, B-D-E, C-D-E)

- Nodes with high betweenness centrality have influence in the network by virtue of their control over information passing between others
 - They get to see the messages as they pass through
 - They could get paid for passing the message along
- Thus, they get a lot of power: their removal would disrupt communication

➤ Closeness Centrality:

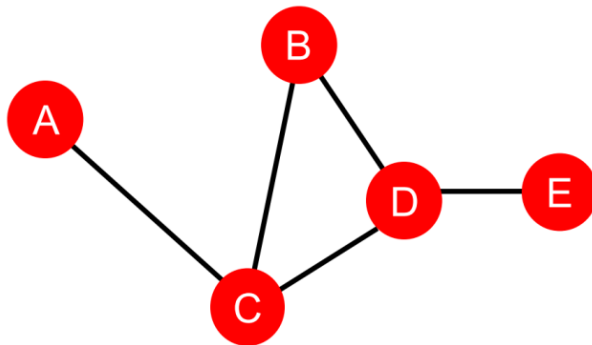
- A node is important if it has **small shortest path lengths** to all other nodes

$$c_v = \frac{1}{\sum_{u \neq v} \text{shortest path length between } u \text{ and } v}$$

- Usually normalized by:

$$\bar{c}(v_i) = \frac{n-1}{\sum_{j=1}^n |d(v_i, v_j)|}$$

- For example:



$$\bullet C_A = 1/(2 + 1 + 2 + 3) = 1/8$$

(A-C-B, A-C, A-C-D, A-C-D-E)

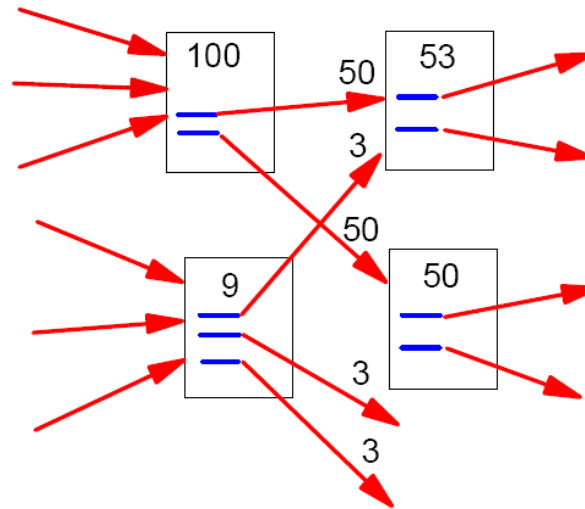
$$\bullet C_D = 1/(2 + 1 + 1 + 1) = 1/5$$

(D-C-A, D-B, D-C, D-E)

- PageRank is a numeric value that represents how important a page is on the web
- Webpage importance
 - One page links to another page = A vote for the other page A link from page A to page B is a vote on A to B
 - If page A is more important itself, then the vote of A to B should carry more weight
 - More votes = More important the page must be

➤ Importance Computation

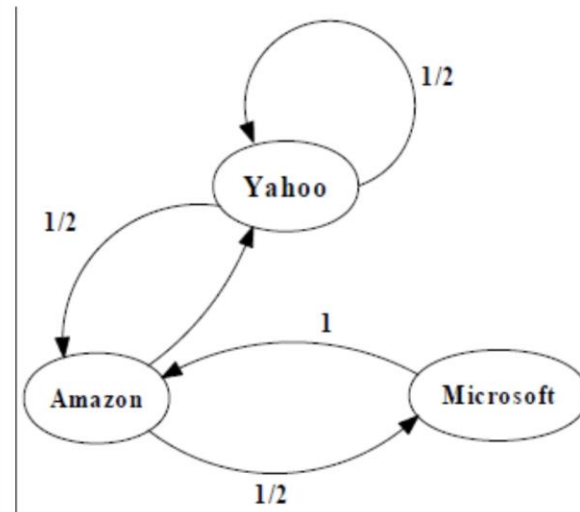
- The importance of a page is distributed to pages that it points to
- The importance of a page is the aggregation of the importance shares of the pages that points to it
 - If a page has 5 outlinks, the importance of the page is divided into 5 and each link receives one fifth share of the importance



$$R(u) = c \sum_{v \in B_u} \frac{R(v)}{N_v}$$

- u : a web page
- B_u : the set of u 's backlinks
- N_v : the number of forward links of page v
- c : the normalization factor to make $\|R\|_{L1} = 1$ ($\|R\|_{L1} = |R_1 + \dots + R_n|$)

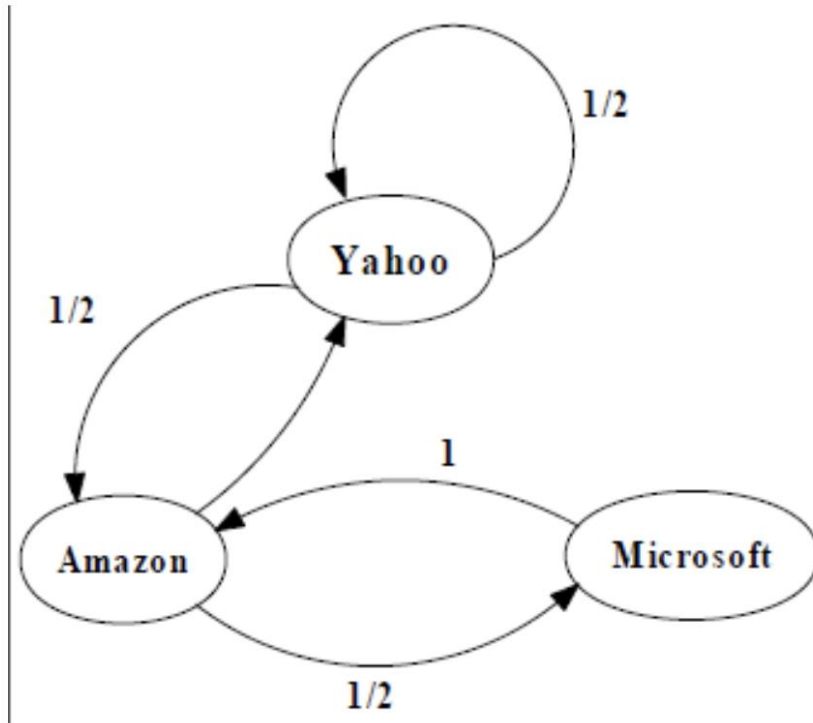
➤ For example:



$$M = \begin{bmatrix} 1/2 & 1/2 & 0 \\ 1/2 & 0 & 1 \\ 0 & 1/2 & 0 \end{bmatrix}$$

$$\begin{bmatrix} \text{yahoo} \\ \text{Amazon} \\ \text{Microsoft} \end{bmatrix} = \begin{bmatrix} 1/3 \\ 1/3 \\ 1/3 \end{bmatrix}$$

$$\begin{bmatrix} 1/3 \\ 1/2 \\ 1/6 \end{bmatrix} = \begin{bmatrix} 1/2 & 1/2 & 0 \\ 1/2 & 0 & 1 \\ 0 & 1/2 & 0 \end{bmatrix} \begin{bmatrix} 1/3 \\ 1/3 \\ 1/3 \end{bmatrix}$$

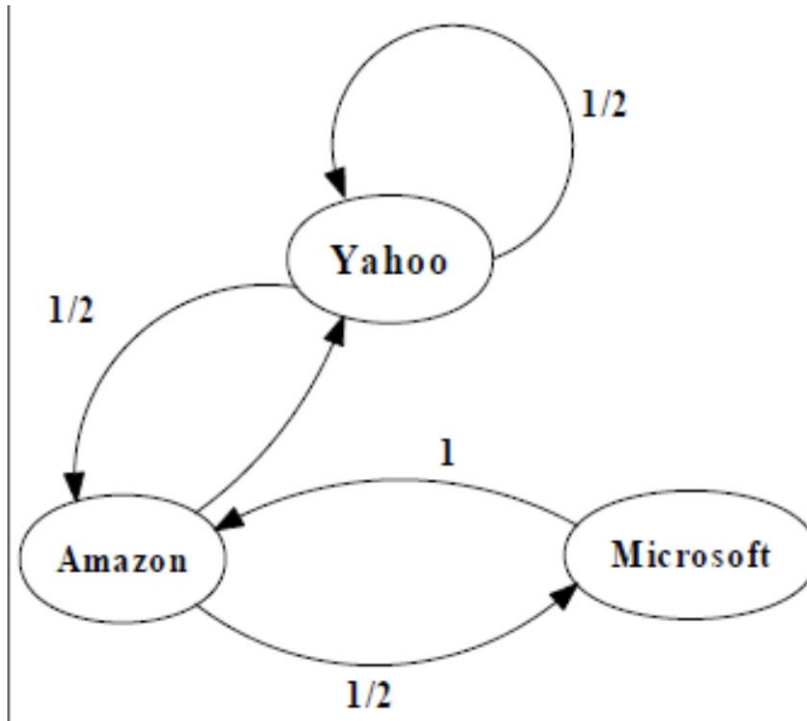


$$M = \begin{bmatrix} 1/2 & 1/2 & 0 \\ 1/2 & 0 & 1 \\ 0 & 1/2 & 0 \end{bmatrix}$$

$$\begin{bmatrix} \text{yahoo} \\ \text{Amazon} \\ \text{Microsoft} \end{bmatrix} = \begin{bmatrix} 1/3 \\ 1/3 \\ 1/3 \end{bmatrix}$$

$$\begin{bmatrix} 5/12 \\ 1/3 \\ 1/4 \end{bmatrix} = \begin{bmatrix} 1/2 & 1/2 & 0 \\ 1/2 & 0 & 1 \\ 0 & 1/2 & 0 \end{bmatrix} \begin{bmatrix} 1/3 \\ 1/2 \\ 1/6 \end{bmatrix}$$

PageRank Calculation: second iteration



$$M = \begin{bmatrix} 1/2 & 1/2 & 0 \\ 1/2 & 0 & 1 \\ 0 & 1/2 & 0 \end{bmatrix}$$

$$\begin{bmatrix} \text{yahoo} \\ \text{Amazon} \\ \text{Microsoft} \end{bmatrix} = \begin{bmatrix} 1/3 \\ 1/3 \\ 1/3 \end{bmatrix}$$

$$\begin{bmatrix} 3/8 \\ 11/24 \\ 1/6 \end{bmatrix} \quad \begin{bmatrix} 5/12 \\ 17/48 \\ 11/48 \end{bmatrix} \quad \dots \quad \begin{bmatrix} 2/5 \\ 2/5 \\ 1/5 \end{bmatrix}$$

Convergence after some iterations

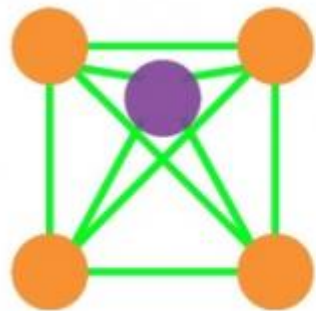
- The clustering coefficient measures how connected a node's neighbours are to one another
- The range is from 0 to 1 (from non-neighbour are connected to each other to all neighbours are fully connected)
- There are three types of clustering coefficient:
 - Local clustering coefficient
 - Average clustering coefficient
 - Global clustering coefficient

- How close its neighbours are to being a clique (complete graph)
- For a node i with degree d_i and L_i represents the number of edges between neighbours of node i

The local clustering coefficient C_i for a node i is defined as:

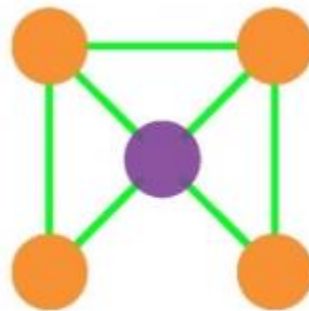
$$C_i = \frac{2L_i}{d_i(d_i - 1)}$$

neighbors of node i form
a complete graph

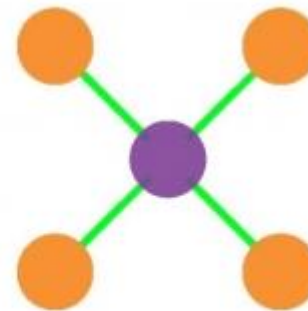


$$C_i = 1$$

50% chance that two neighbors
of a node i are linked



$$C_i = 1/2$$

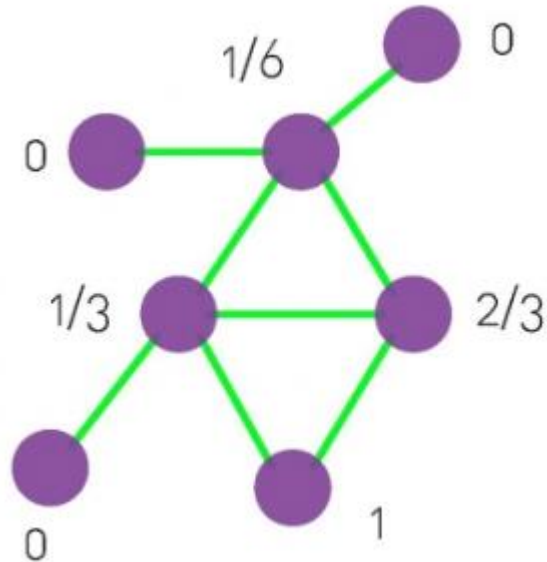


$$C_i = 0$$

None of neighbors of node i
link to each other

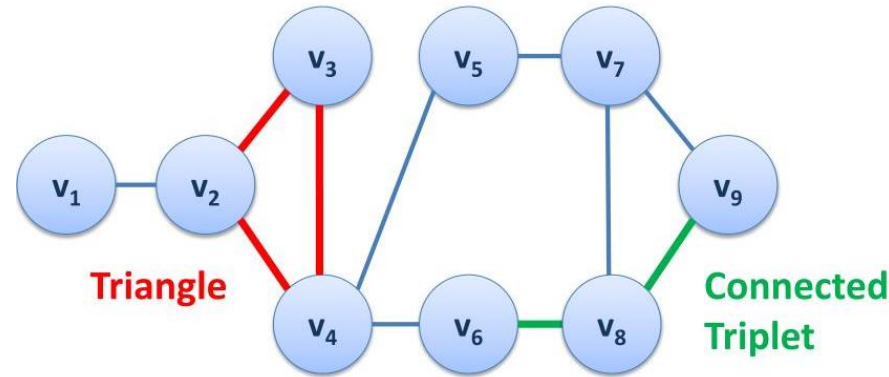
- The degree of clustering of a whole network is captured by the average clustering coefficient, namely $\langle C \rangle$, representing the average of all the local clustering coefficient C_i over all nodes $i = 1, \dots, N$.

$$\langle C \rangle = \frac{1}{N} \sum_{i=0}^N C_i$$



$$\langle C \rangle = \frac{1}{7} * \left(0 + \frac{1}{6} + \frac{1}{3} + \frac{2}{3} + 1 + 0 + 0 \right) = 0.333$$

- The global clustering coefficient is based on triplets of nodes
- A triplet consists of three connected nodes. A triangle therefore includes three closed triplets, one centered on each of the nodes

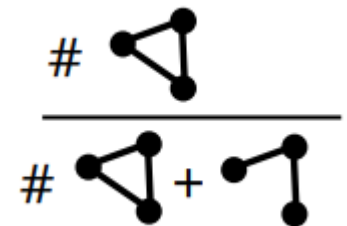


- The global clustering coefficient is the number of closed triplets over the total number of triplets (both open and closed)

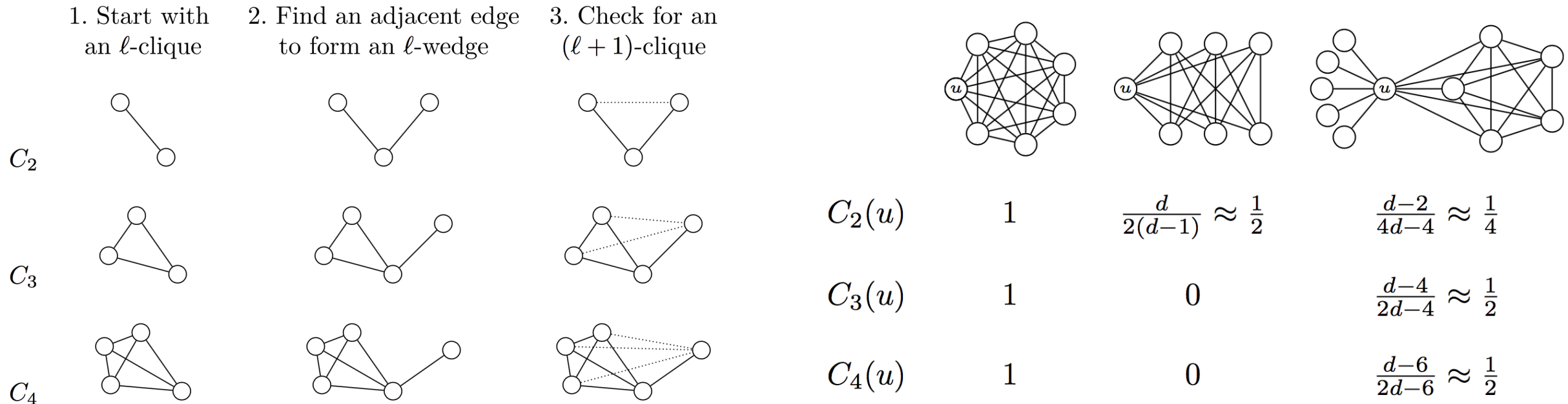
Closed triplets: $(v_2, v_3, v_4), (v_7, v_8, v_9)$

Connected triplets: $(v_6, v_8, v_9), \dots$

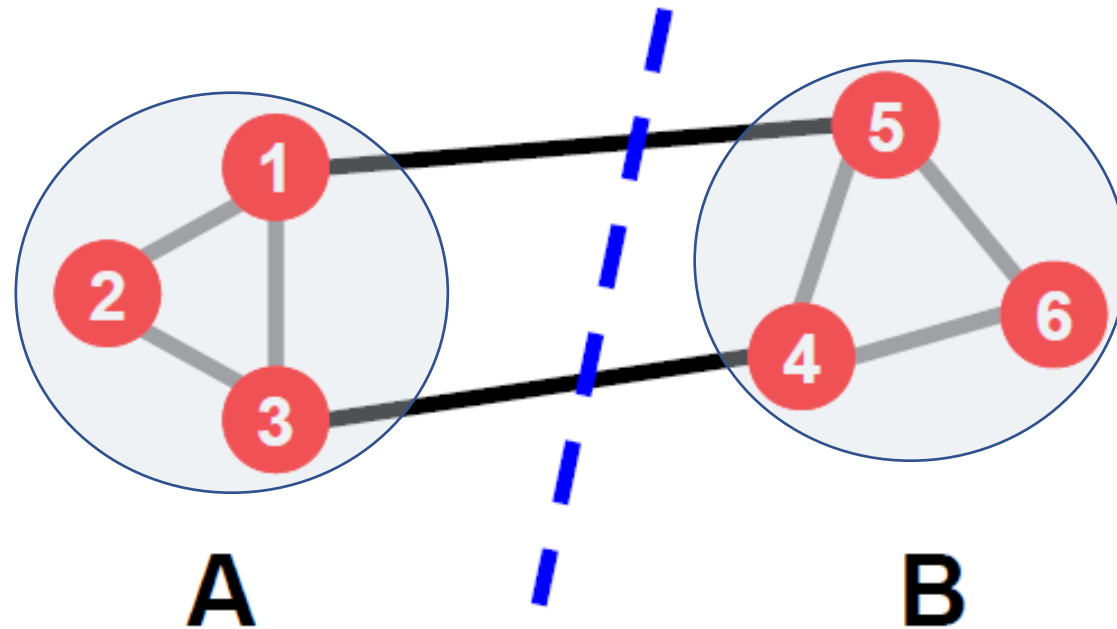
$$C(G) = \frac{\# \text{ of closed triplets}}{\# \text{ of connected triplets}}$$



- The clustering coefficient can be extended to higher order structures with k-cliques

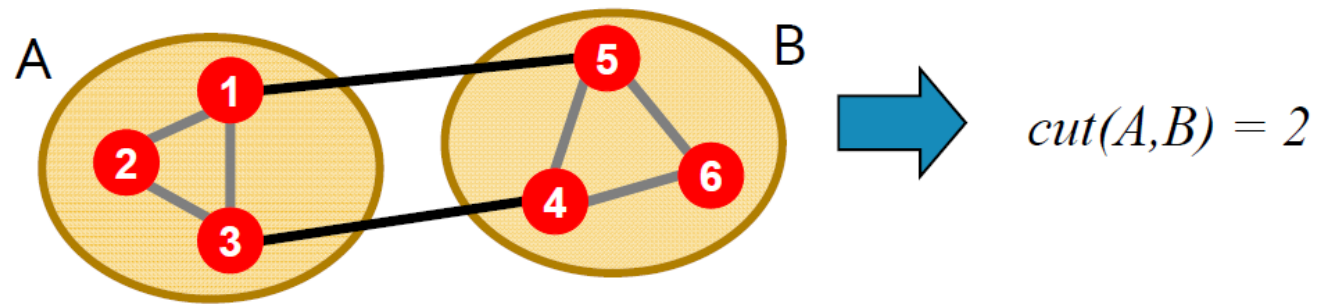


- Basic principle for graph partitioning:
 - Minimize the number of between-group connections
 - Maximize the number of within-group connections



Graph partitioning : A & B

- For considering between-group:



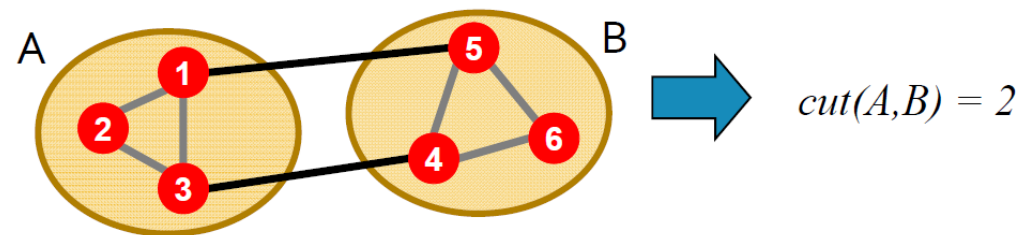
Cut: Set of edges with only one vertex in a group:
 $cut(A,B) = \sum_{i \in A, j \in B} w_{ij}$

- For considering within-group
- $\text{Assoc}(A, V)$: the total connection from nodes in A to all nodes in the graph

$$\text{assoc}(A, V) = \sum_{u \in A, t \in V} w(u, t)$$

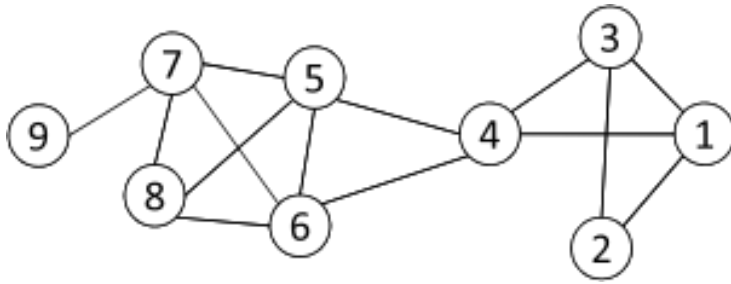
$$\text{Ncut}(A, B) = \frac{\text{cut}(A, B)}{\text{assoc}(A, V)} + \frac{\text{cut}(A, B)}{\text{assoc}(B, V)}$$

$$\text{Conductance}(A, B) = \frac{\text{cut}(A, B)}{\min(\text{assoc}(A, V), \text{assoc}(B, V))}$$



- Modularity measures the strength of a community partition by considering the degree distribution
- Given a graph with m edges, the expected number of edges between two nodes with degree d_i and d_j is

$$d_i d_j / 2m$$



The expected number of edges between nodes 1 and 2 is $3 \cdot 2 / (2 \cdot 14) = 3/14$

- Modularity:

$$Q = \frac{1}{2m} \sum_{ij} (A_{ij} - \gamma \frac{d_i d_j}{2m}) \delta(C_i, C_j)$$

probability a random edge would go between i and j

Adjacency matrix of graph

$\delta(C_i, C_j)$: 1 if i and j are in the same community else 0

- We have modularity matrix:

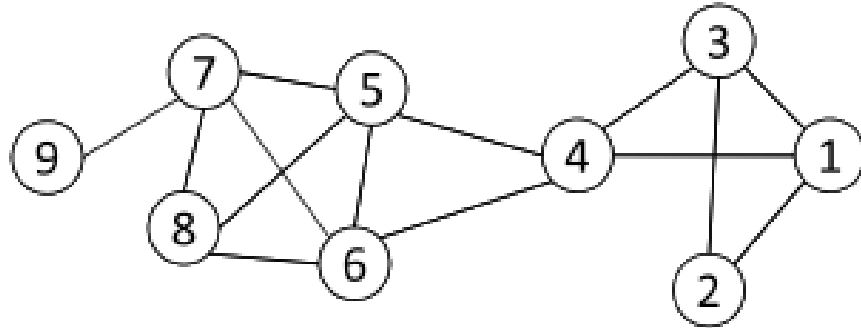
$$B = A - \mathbf{d}\mathbf{d}^T/2m \quad (B_{ij} = A_{ij} - d_i d_j / 2m)$$

Where: d is node degree

- Similar to spectral clustering, modularity maximization can be reformulated as

$$\max Q = \frac{1}{2m} \text{Tr}(S^T B S) \quad s.t. \quad S^T S = I_k$$

- Optimal solution: top eigenvectors of the modularity matrix
- Apply k-means to S as a post-processing step to obtain community partition



Two Communities:
 $\{1, 2, 3, 4\}$ and $\{5, 6, 7, 8, 9\}$

↑
 k-means

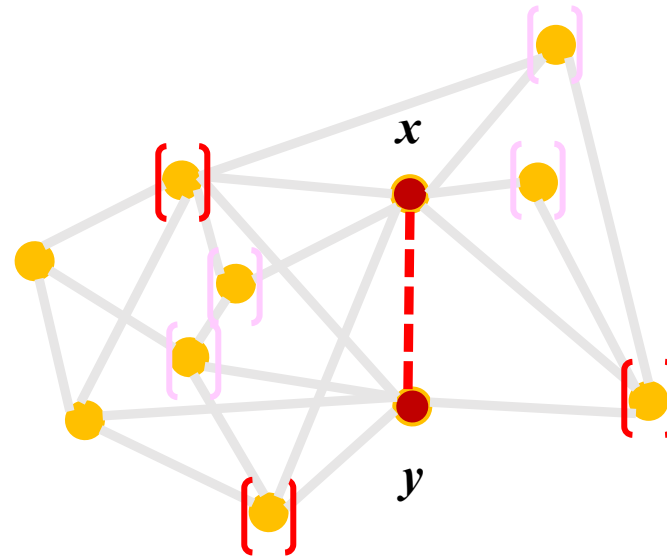
$$B = \begin{bmatrix} -0.32 & 0.79 & 0.68 & 0.57 & -0.43 & -0.43 & -0.43 & -0.32 & -0.11 \\ 0.79 & -0.14 & 0.79 & -0.29 & -0.29 & -0.29 & -0.29 & -0.21 & -0.07 \\ 0.68 & 0.79 & -0.32 & 0.57 & -0.43 & -0.43 & -0.43 & -0.32 & -0.11 \\ 0.57 & -0.29 & 0.57 & -0.57 & 0.43 & 0.43 & -0.57 & -0.43 & -0.14 \\ -0.43 & -0.29 & -0.43 & 0.43 & -0.57 & 0.43 & 0.43 & 0.57 & -0.14 \\ -0.43 & -0.29 & -0.43 & 0.43 & 0.43 & -0.57 & 0.43 & 0.57 & -0.14 \\ -0.43 & -0.29 & -0.43 & -0.57 & 0.43 & 0.43 & -0.57 & 0.57 & 0.86 \\ -0.32 & -0.21 & -0.32 & -0.43 & 0.57 & 0.57 & 0.57 & -0.32 & -0.11 \\ -0.11 & -0.07 & -0.11 & -0.14 & -0.14 & -0.14 & 0.86 & -0.11 & -0.04 \end{bmatrix}$$

Modularity Matrix



$$S = \begin{bmatrix} 0.44 & -0.00 \\ 0.38 & 0.23 \\ 0.44 & -0.00 \\ 0.17 & -0.48 \\ -0.29 & -0.32 \\ -0.29 & -0.32 \\ -0.38 & 0.34 \\ -0.34 & -0.08 \\ -0.14 & 0.63 \end{bmatrix}$$

- Jaccard coefficient:
 - How likely a neighbour of x is also a neighbour of y
 - Same as common neighbours, adjusted for degree



$$JC = \frac{|N(x) \cap N(y)|}{|N(x) \cup N(y)|} = \frac{CN}{d_x + d_y - CN}$$

➤ Katz index:

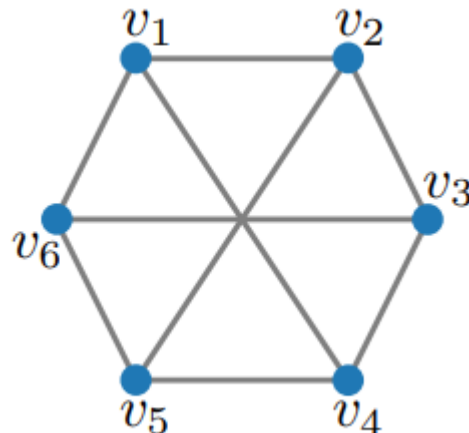
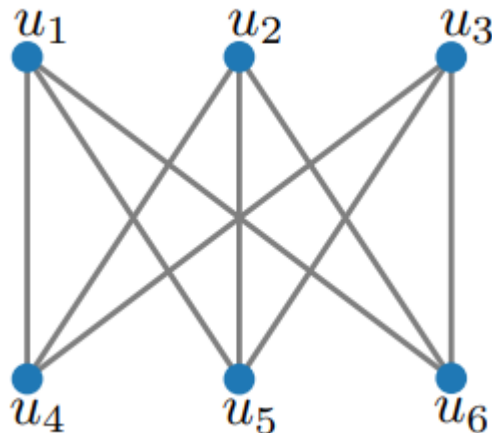
Element (x,y) in the adjacency matrix

$$score(x, y) = \sum_{l=1}^{\infty} \beta^l |paths_{xy}^{(l)}| = \beta A_{xy} + \beta^2 A_{xy}^2 + \dots$$

- Sum over ALL paths of length ℓ
- $0 < \beta < 1$ is a parameter of the predictor, exponentially damped to count short paths more heavily
- Small damped parameter = predictions much like common neighbours

➤ Graph isomorphism:

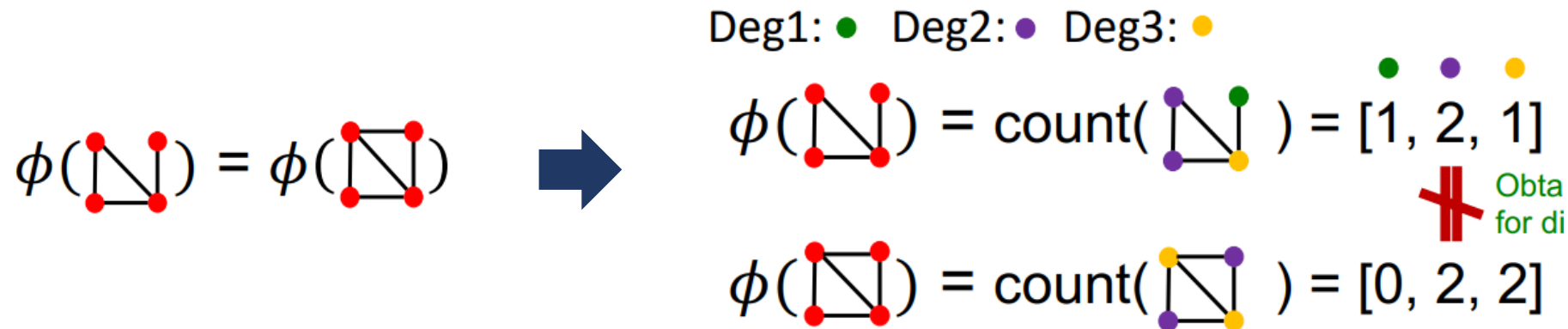
- Find a mapping f of the nodes of G_1 to the nodes of G_2 such that G_1 and G_2 are identical
- i.e. (u_i, u_j) is an edge of G_1 iff $(f(u_i), f(u_j))$ is an edge of G_2 . Then f is an isomorphism, and G_1 and G_2 are called isomorphic
- No polynomial-time algorithm is known for graph isomorphism
- Neither is it known to be NP-complete



$$\begin{array}{cc} V^{G_1} & V^{G_2} \\ \psi : & \\ v_1 & \rightarrow u_1 \\ v_2 & \rightarrow u_4 \\ v_3 & \rightarrow u_2 \\ v_4 & \rightarrow u_5 \\ v_5 & \rightarrow u_3 \\ v_6 & \rightarrow u_6 \end{array}$$

- Kernel is a type of measures of similarity
- Mapping two objects x and x' via mapping $\boxed{\phi}$ into feature space H
- Measure their similarity in H as $\langle \phi(x), \phi(x') \rangle$.
- **Kernel Trick**: Compute inner product in H as kernel in input space

$$\boxed{k(x, x') = \langle \phi(x), \phi(x') \rangle.}$$

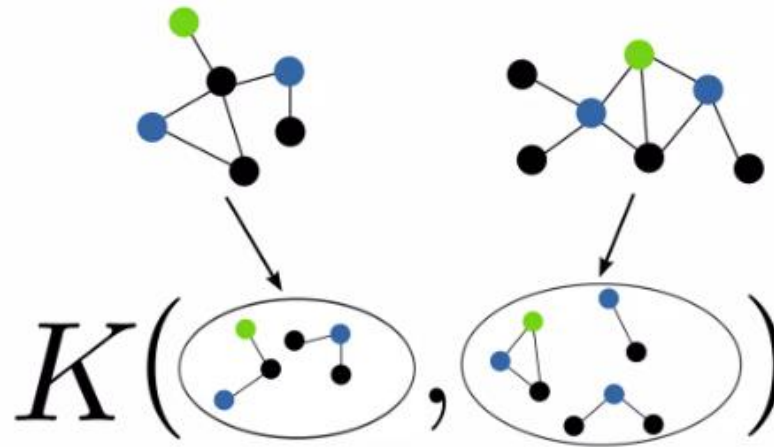


- Instance of R-convolution kernels by Haussler (1999):
 - R-convolution kernels **compare decompositions of two structured objects**

$$k_{convolution}(x, x') = \sum_{(x_d, x) \in \mathcal{R}} \sum_{(x'_d, x') \in \mathcal{R}} k_{parts}(x_d, x'_d)$$

- **Decompose graphs into their substructures and add up the pairwise similarities** between these substructures
- Concept:
 - Kernel function to measure the similarity of pairs of graphs by computing an inner product on graphs
 - Compare substructures of graphs that are computable in polynomial time

- Graph kernels based on **bags of patterns**:
 - Extraction of a set of patterns from graphs
 - Comparison between patterns
 - Comparison between bags of patterns



- Graph kernels are one of the most recent approaches to graph comparison
- Interestingly, graph kernels employ concepts from all three traditional branches of graph comparison:
 - Measure similarity in terms of **isomorphic substructures of graphs**
 - **Allow for inexact matching of nodes, edges, and labels**
 - Treat graphs as **vectors** in a Hilbert space of graph features

Floyd-transformation:

- Given an input graph G , outputs a **shortest-path graph S**
 - S contains the **same set of nodes** as the input graph G
 - There exists an edge between all nodes in S which are **connected by a path in G**
 - Every edge in S between two nodes is **labelled by the shortest distance** between these two nodes
 - This transformation can be done in **$O(n^3)$** time

- Compute all-pairs-shortest-paths for G and G' via Floyd-Warshall
- Define a kernel by comparing all pairs of shortest path lengths from G and G' :

$$k(G, G') = \sum_{v_i, v_j \in G} \sum_{v'_k, v'_l \in G'} k_{length}(d(v_i, v_j), d(v'_k, v'_l))$$

where $d(v_i, v_j)$ is the length of the shortest path between node v_i and v_j

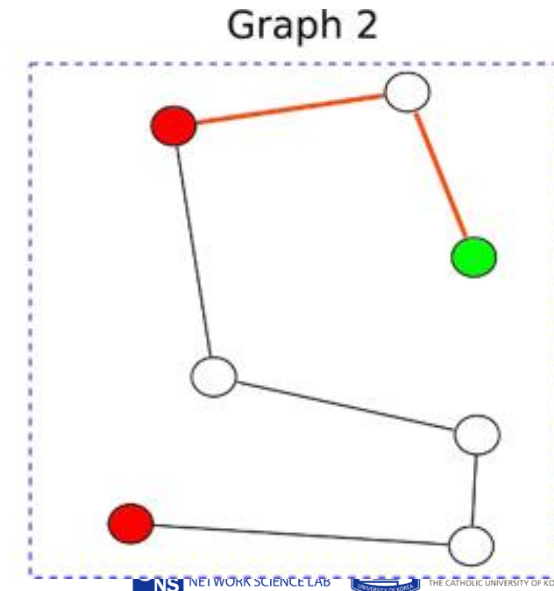
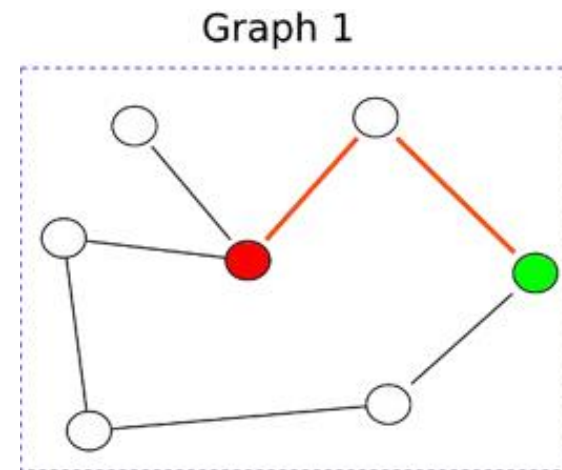
- k_{length} is a kernel that compares the lengths of two shortest paths:

- a linear kernel

$$k(d(v_i, v_j), d(v'_k, v'_l)) = d(v_i, v_j) * d(v'_k, v'_l)$$

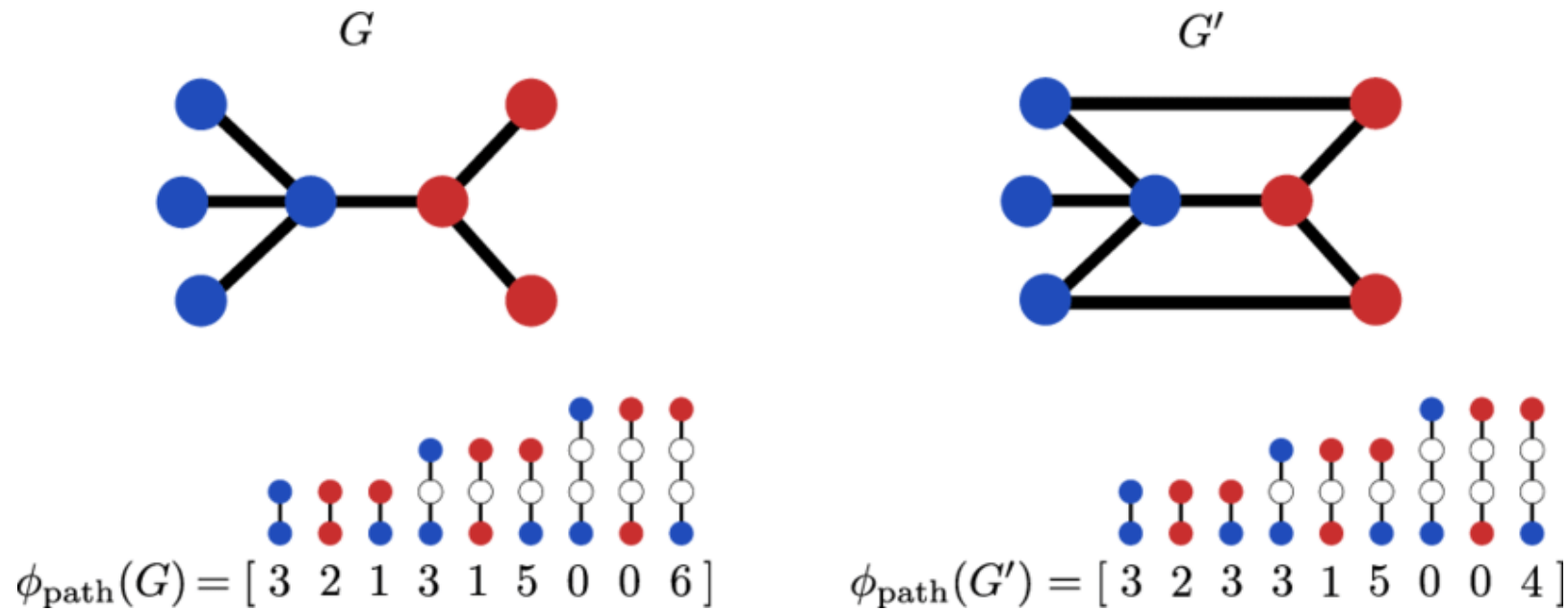
- a delta kernel

$$k(d(v_i, v_j), d(v'_k, v'_l)) = \begin{cases} 1, & \text{if } d(v_i, v_j) = d(v'_k, v'_l) \\ 0, & \text{otherwise} \end{cases}$$



Shortest-path graph kernel:

$$k(G, G') = \sum_{v_i, v_j \in G} \sum_{v'_k, v'_l \in G'} k_{length}(d(v_i, v_j), d(v'_k, v'_l))$$

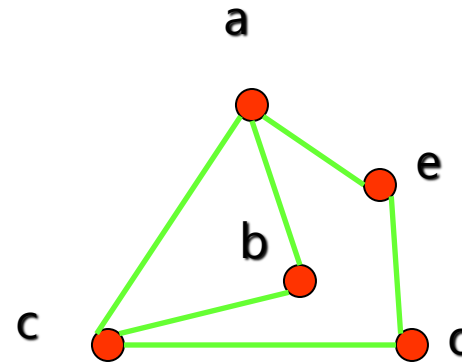
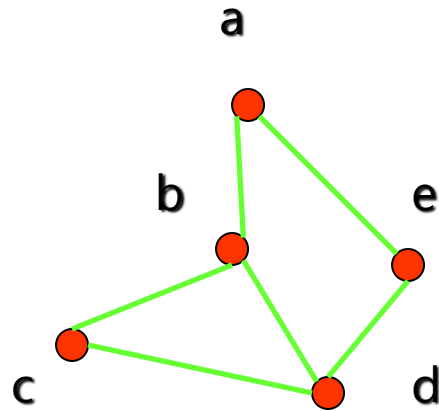


Definition:

- The simple graphs $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$ are isomorphic if there is a bijection (a one-to-one and onto function) f from V_1 to V_2 with the property that a and b are adjacent in G_1 if and only if $f(a)$ and $f(b)$ are adjacent in G_2 , for all a and b in V_1
- Such a function f is called an isomorphism
- In other words, G_1 and G_2 are isomorphic if their nodes can be ordered in such a way that the adjacency matrices $M(G_1)$ and $M(G_2)$ are identical

- For this purpose, we can check invariants, that is, properties that two isomorphic simple graphs must both have
- For example, they must have
 - The same number of nodes,
 - The same number of edges,
 - And the same degrees of nodes
- Note that two graphs that differ in any of these invariants are not isomorphic, but two graphs that match in all of them are not necessarily isomorphic

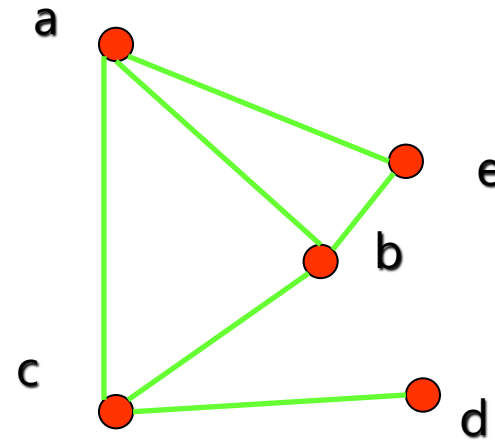
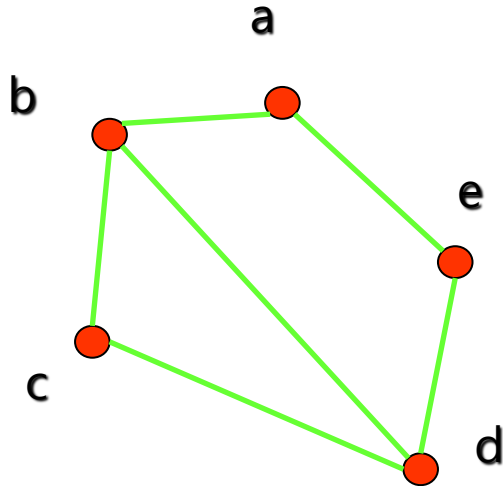
- Are the following two graphs isomorphic?



- **Solution:** Yes, they are isomorphic, because they can be arranged to look identical
- You can see this if in the right graph you move vertex b to the left of the edge {a, c}. Then the isomorphism f from the left to the right graph is

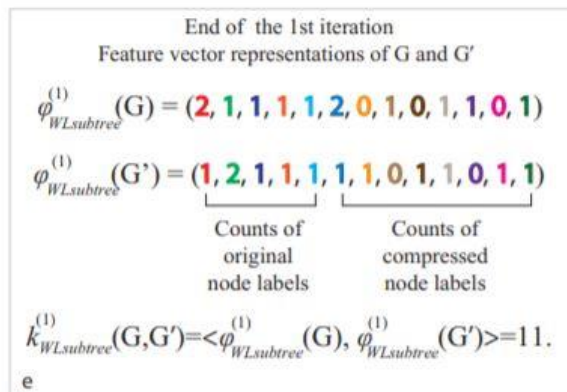
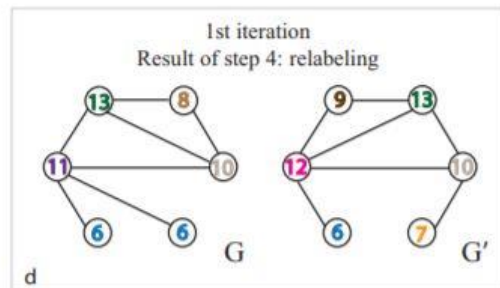
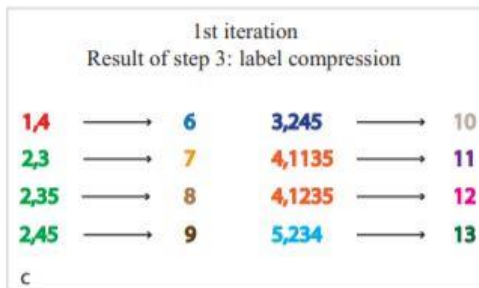
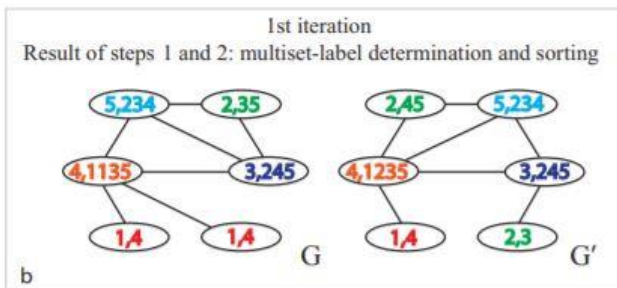
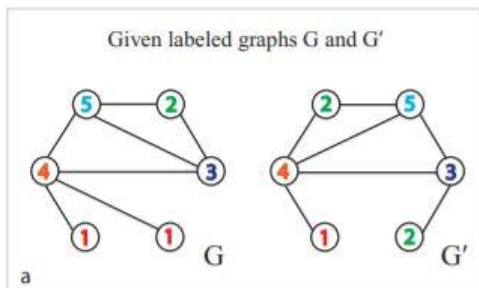
$$f(a) = e, f(b) = a, f(c) = b, f(d) = c, f(e) = d$$

- Are the following two graphs isomorphic?



- **Solution:** No, they are not isomorphic, because they **differ in the degrees of their nodes**
- Node d in right graph is of degree one, but there is no such node in the left graph

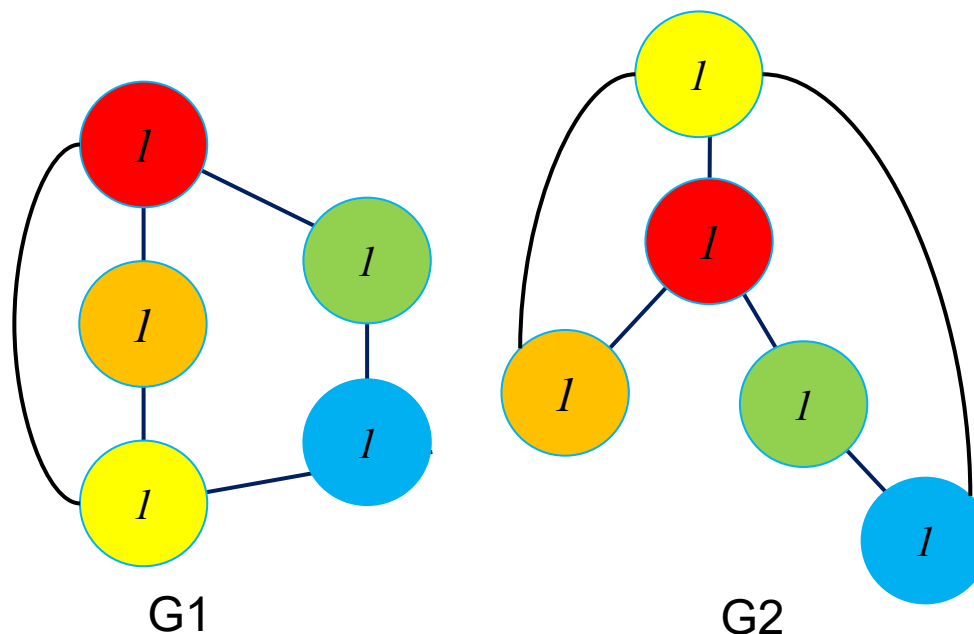
➤ Weisfeiler-Lehman Isomorphism Testing:



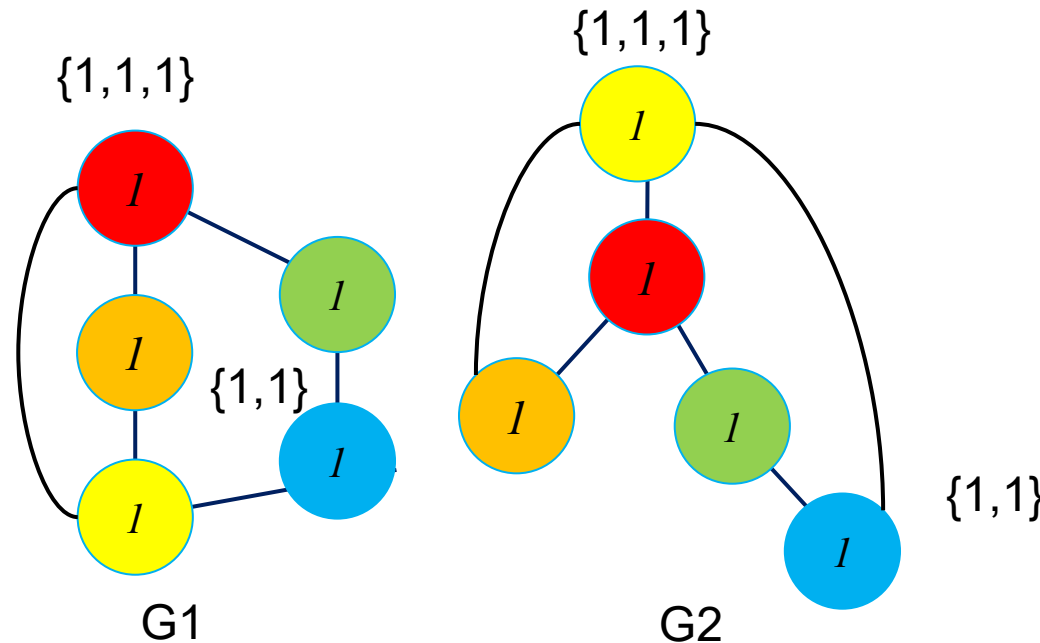
Algorithm 1: WL-1 algorithm (Weisfeiler & Lehmann, 1968)

Input: Initial node coloring $(h_1^{(0)}, h_2^{(0)}, \dots, h_N^{(0)})$
Output: Final node coloring $(h_1^{(T)}, h_2^{(T)}, \dots, h_N^{(T)})$
 $t \leftarrow 0$;
repeat
 for $v_i \in \mathcal{V}$ **do**
 $h_i^{(t+1)} \leftarrow \text{hash}(\sum_{j \in \mathcal{N}_i} h_j^{(t)})$;
 $t \leftarrow t + 1$;
until *stable node coloring is reached*;

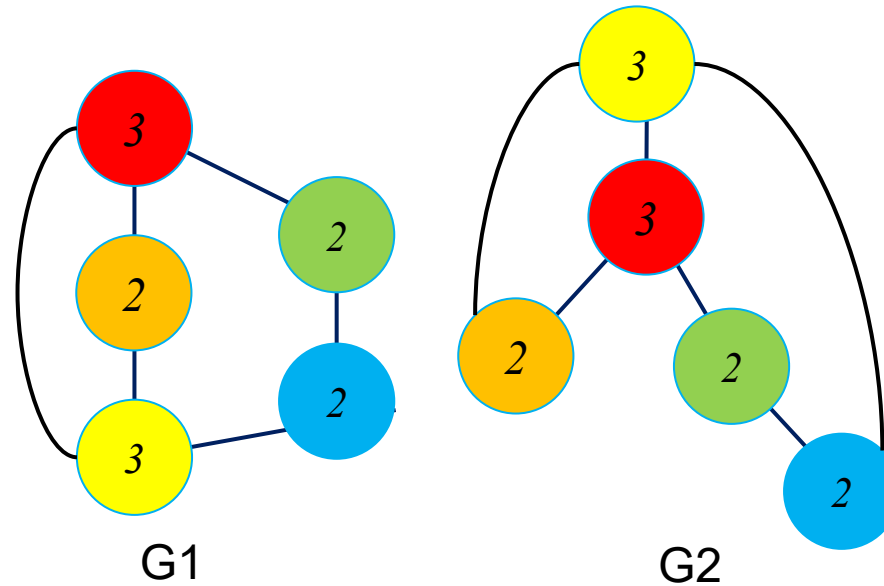
- We will apply the Weisfeiler-Lehman isomorphism test to these graphs as a means of illustrating the test
- Step 1: Set node label =1 for all nodes



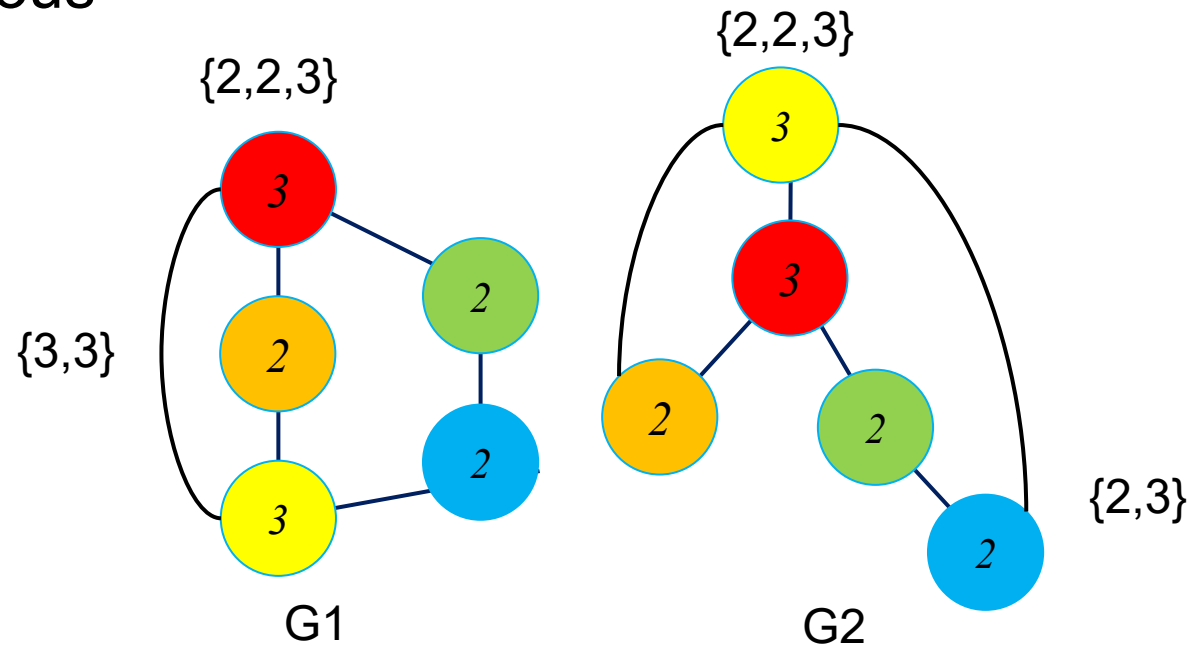
- We will apply the Weisfeiler-Lehman isomorphism test to these graphs as a means of illustrating the test
- Step 1: Set node label = 1 for all nodes
- Step 2: Compute multiset of the neighbouring nodes' compressed labels



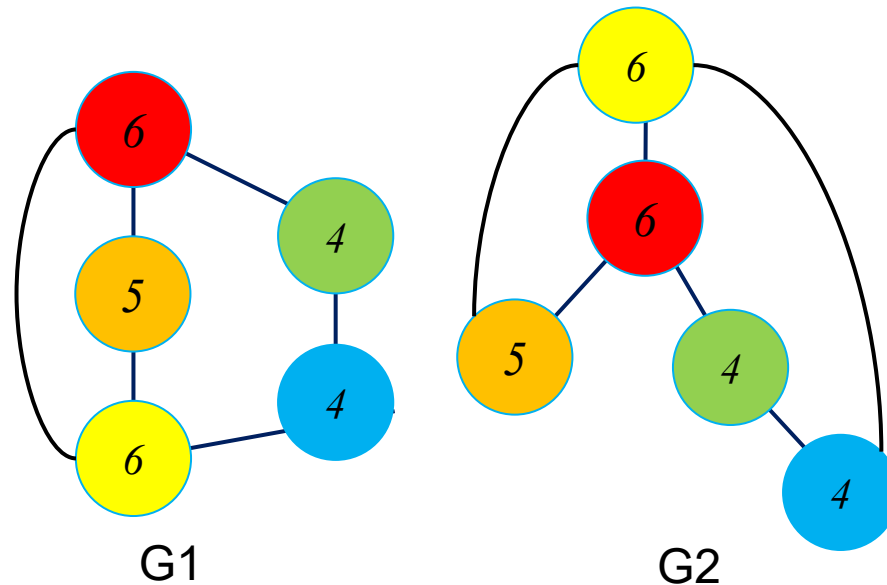
- We will apply the Weisfeiler-Lehman isomorphism test to these graphs as a means of illustrating the test
- Step 1: Set node label =1 for all nodes
- Step 2: Compute multiset of the neighbouring nodes' compressed labels



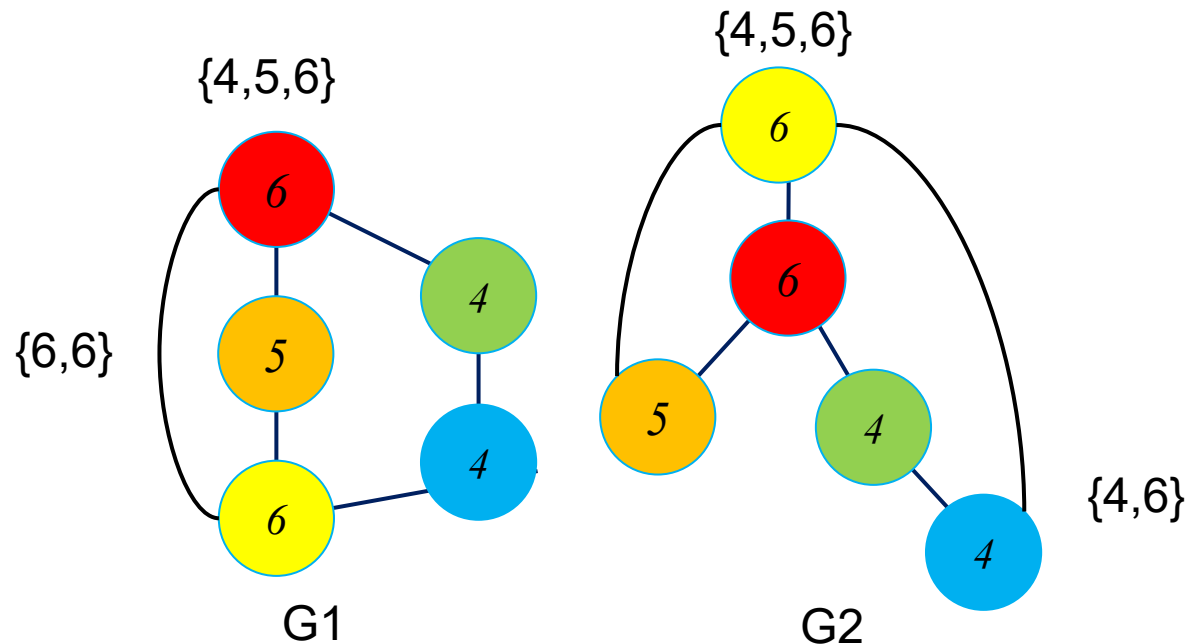
- We will apply the Weisfeiler-Lehman isomorphism test to these graphs as a means of illustrating the test
- Step 1: Set node label = 1 for all nodes
- Step 2: Compute multiset of the neighbouring nodes' compressed labels
- Step 3: Continuous



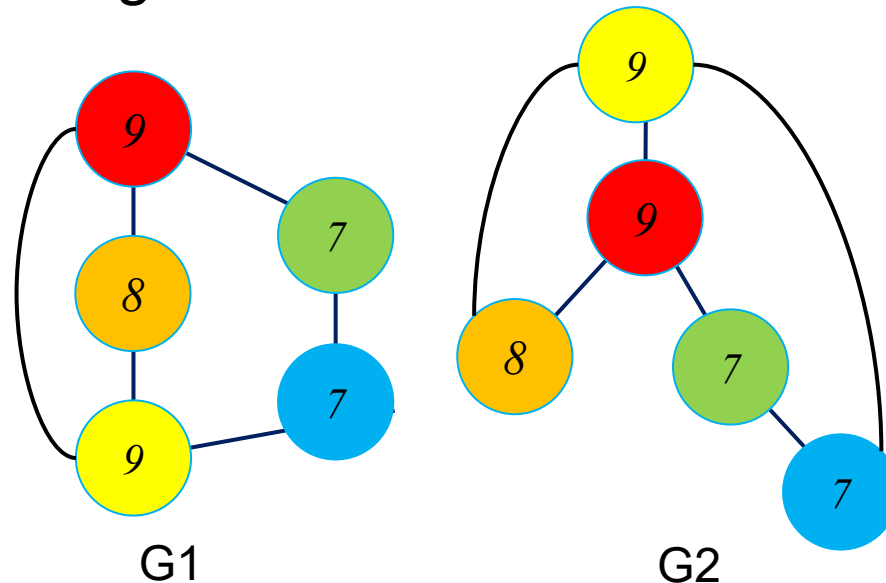
- We will apply the Weisfeiler-Lehman isomorphism test to these graphs as a means of illustrating the test
- Step 1: Set node label =1 for all nodes
- Step 2: Compute multiset of the neighbouring nodes' compressed labels
- Step 3: Continuous....



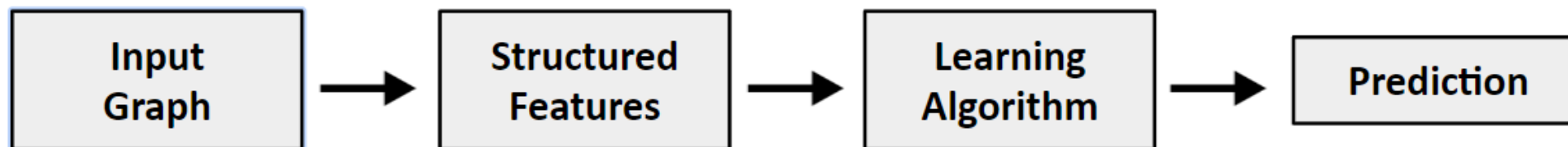
- We will apply the Weisfeiler-Lehman isomorphism test to these graphs as a means of illustrating the test
- Step 1: Set node label = 1 for all nodes
- Step 2: Compute multiset of the neighbouring nodes' compressed labels
- Step 3: Continuous...



- We will apply the Weisfeiler-Lehman isomorphism test to these graphs as a means of illustrating the test
- Step 1: Set node label =1 for all nodes
- Step 2: Compute multiset of the neighboring nodes' compressed labels
- Step 3: Continuous....
- Step 4: Since the partition of nodes by compressed label has not changed, we may terminate the algorithm here



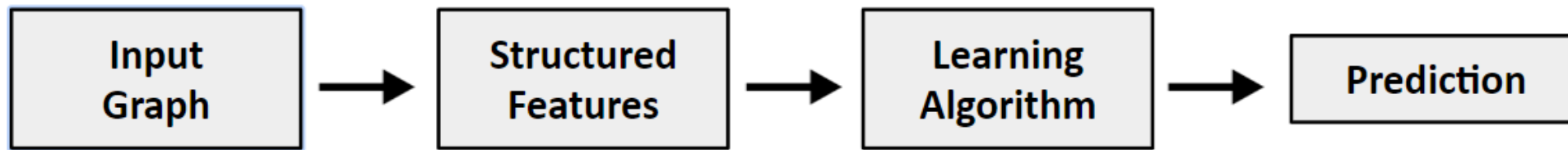
- Given a graph, we can extract node, edge, graph-level features from the graph, then learn a model to map the features to the desired labels



Feature Engineering

- Node feature
- Edge feature
- Graph feature
- SVM
- Random Forest
- XGBoost
- DNN
- Node-level
- Edge-level
- Graph-level

- GRL aims to generate graph representation vectors that describe graph's structure
- We don't need to do feature engineering **every single time**



Feature Engineering

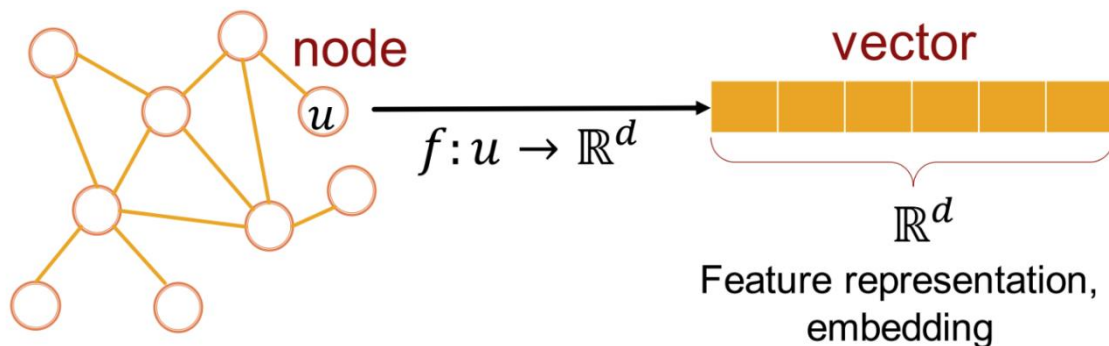


Representation Learning

learn the features by itself

- SVM
- Random Forest
- XGBoost
- DNN
- Node-level
- Edge-level
- Graph-level

- GRL's **goal**: Learn efficient task-independent feature for ML with graphs
 - Map nodes into an embedding space
- Similarity of embeddings between nodes indicates their similarity in the network
 - Example: both nodes are close to each other (connected by an edge)
- For simplicity, no node features or extra information is used



Tasks

- Node classification/regression
- Edge prediction
- Graph classification/regression.
- Graph clustering
- ...

- Goal: Encode nodes $\rightarrow$ similarity in embedding space (dot product) $\approx$ similarity in the original graph

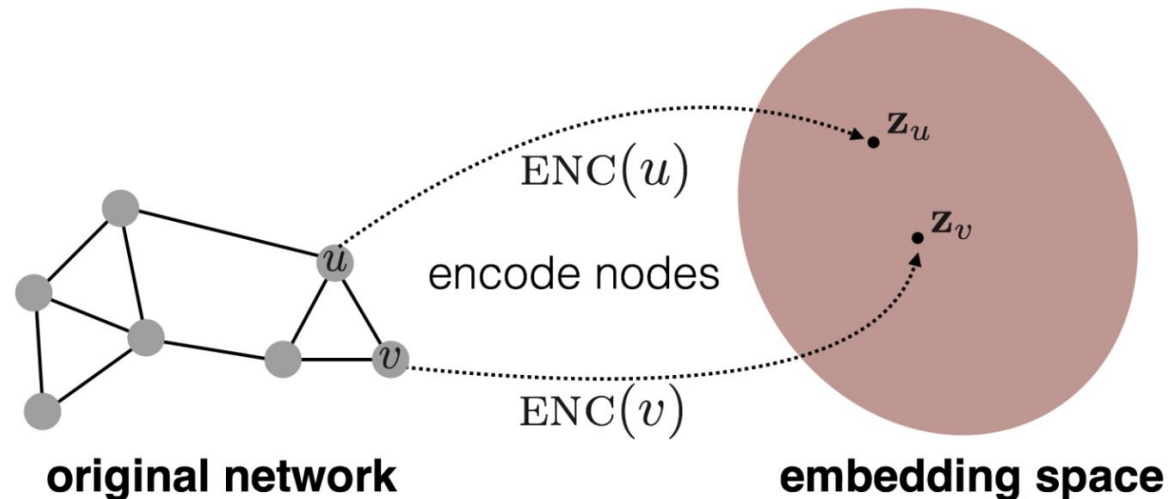
$$\text{similarity}(u, v) \approx \mathbf{z}_v^T \mathbf{z}_u$$

in the original network Similarity of the embedding

We need to define:

$\text{ENC}(u)$

$\text{Similarity}(u, v)$

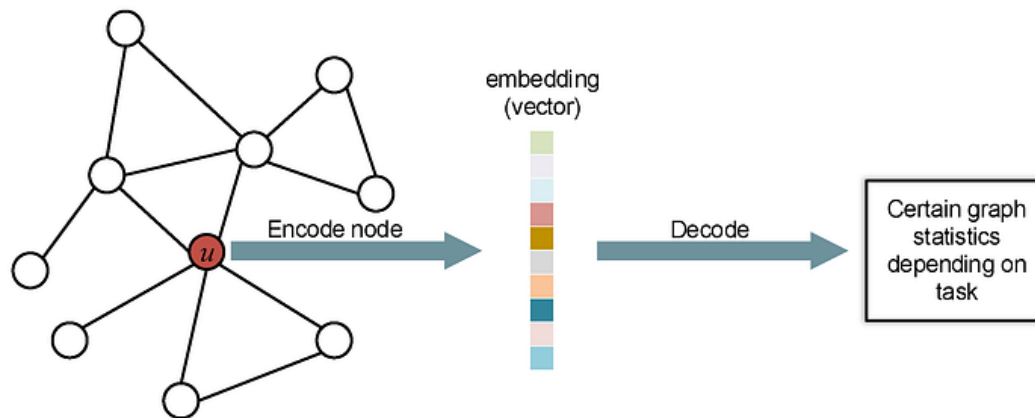


- Encoder: Node $\rightarrow$ embedding (d-dimensional vector, $\text{ENC}(v) = Z_v$)
- Decoder: Map embedding $\rightarrow$ similarity score (dot product)
- Define a node similarity function:
 - Whether 2 nodes are close in the graph, and how close are they?

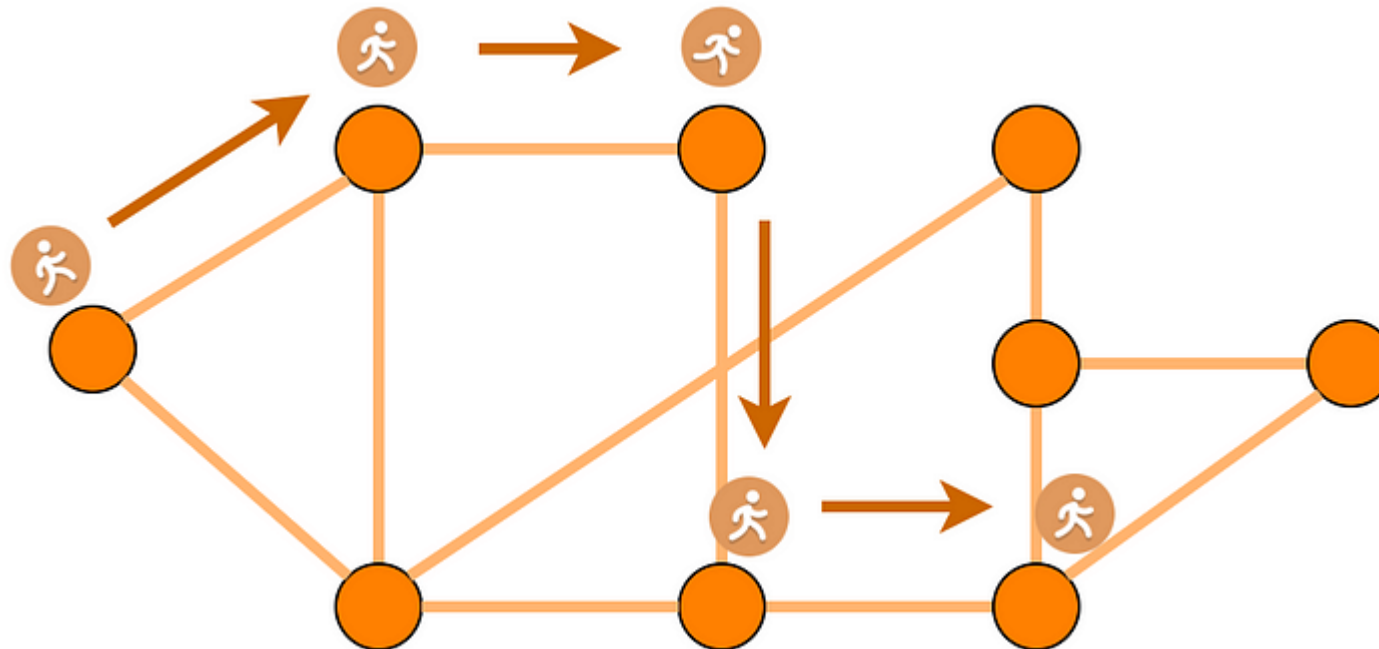
- Optimize $\rightarrow$ $\text{similarity}(u, v) \approx \mathbf{z}_v^T \mathbf{z}_u$
 - in the original network
 - Similarity of the embedding

Dot product between node embedding

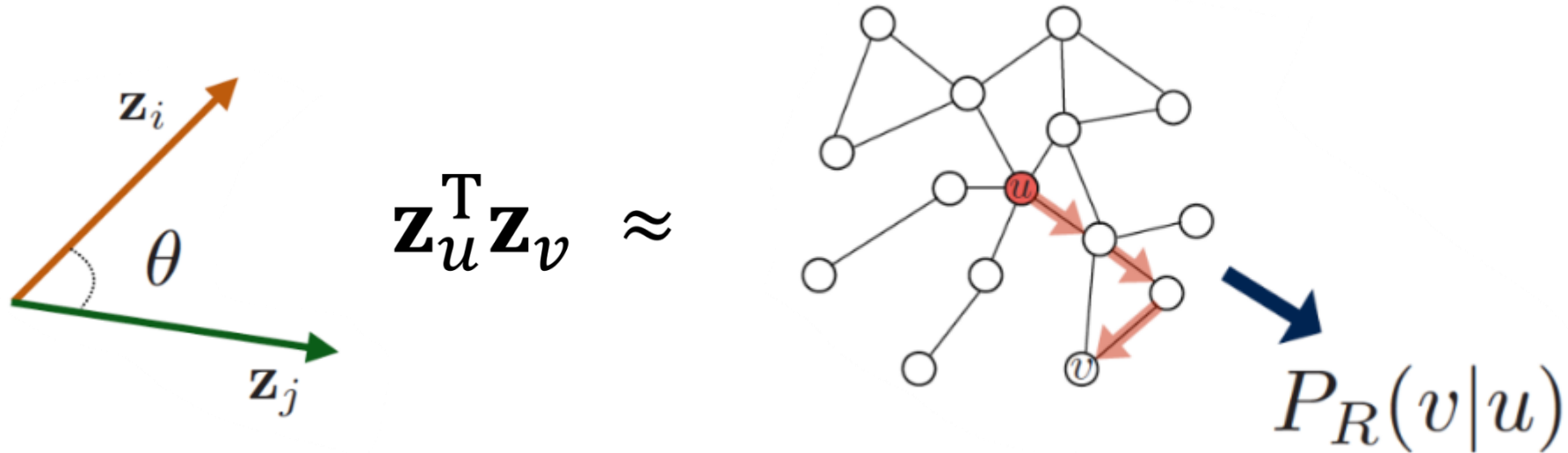
- Maximize $\mathbf{z}_v^T \mathbf{z}_u$ for node pairs (u, v) that are **similar**



- Given a graph and a starting point, we select a neighbour of it at random, and move to this neighbour
- Then, we select a neighbor of this point at random, and move to it,...
- The random sequence of nodes visited this way is **a random walk on the graph**



- Estimate probability of visiting node v on a random walk starting from node u using some random walk strategy R
- Optimize embeddings to encode these random walk statistics



- Run short **random walks starting from each node** on the graph using some strategy R
- For each node u collect **N(u), the multiset* of nodes visited on random walks starting from u**
- Optimize embeddings to according to:

$$\mathcal{L} = \sum_{u \in V} \sum_{v \in N_R(u)} -\log(P(v|\mathbf{z}_u))$$

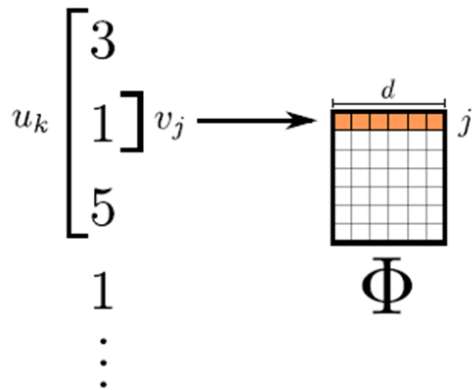
- * N(u) can have repeat elements since nodes can be visited multiple times on random walks

- Employ **short random walks** on the graph to discover the structure

$v_4 \rightarrow v_3 \rightarrow v_1 \rightarrow v_5 \rightarrow v_1 \rightarrow v_{46}$

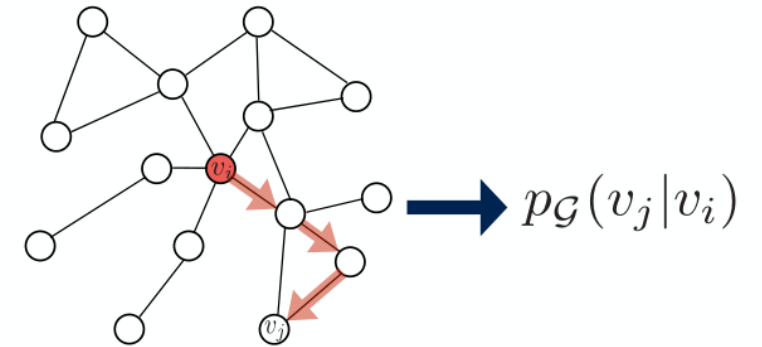
**Random walks in Network
=
Sentences in NLP**

The quick brown fox jumps over the lazy dog.

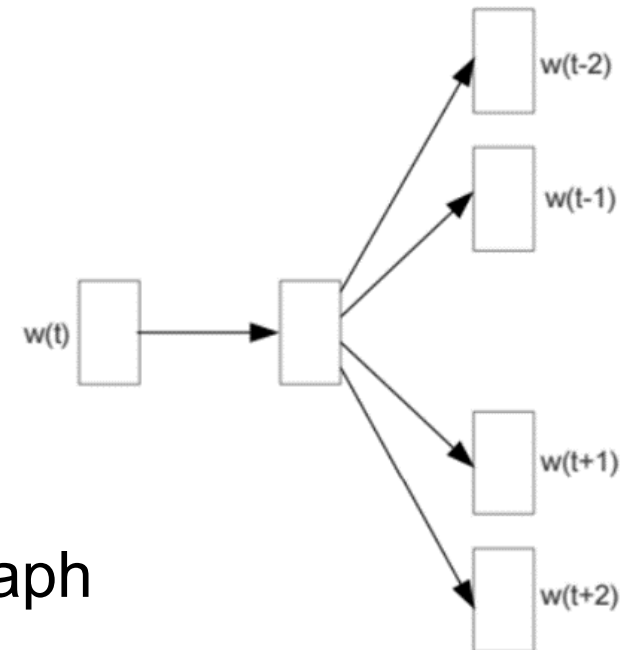


**Node sequence as the input of
word2vec models**

$$\Phi: v \in V \mapsto \mathbb{R}^{|V| \times d}$$



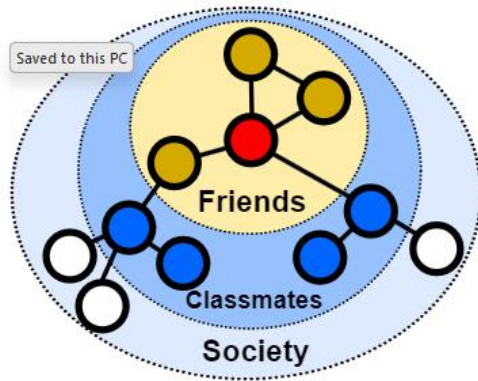
1. Run random walks to obtain co-occurrence statistics.



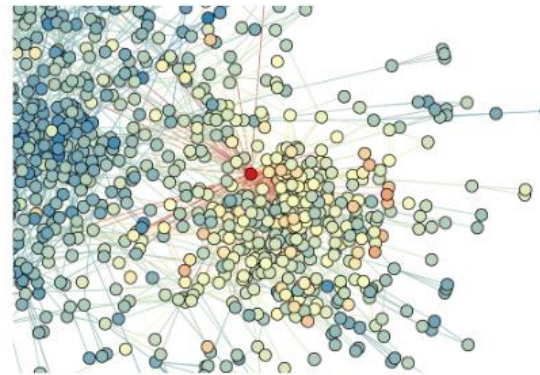
Short random walk is not sufficient in large scale graph

Don't Walk – Skip! Online Learning of Multi-scale Network Embeddings⁵⁵

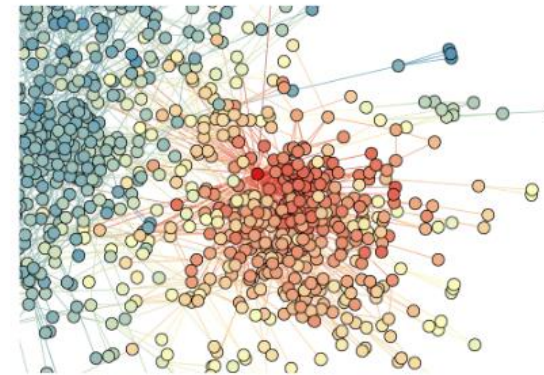
- **Problem:** DeepWalk learn a single global representation that conflates information across all scales
 - Not sufficient in large scale graph
- **Idea:** Multiple Scales of Random Walks at different scales to explicitly deal with large graph
- **Solutions:** Propose WalkLets algorithm with skipping mechanism



(a) A student (in red) is a member of several increasing larger social communities.

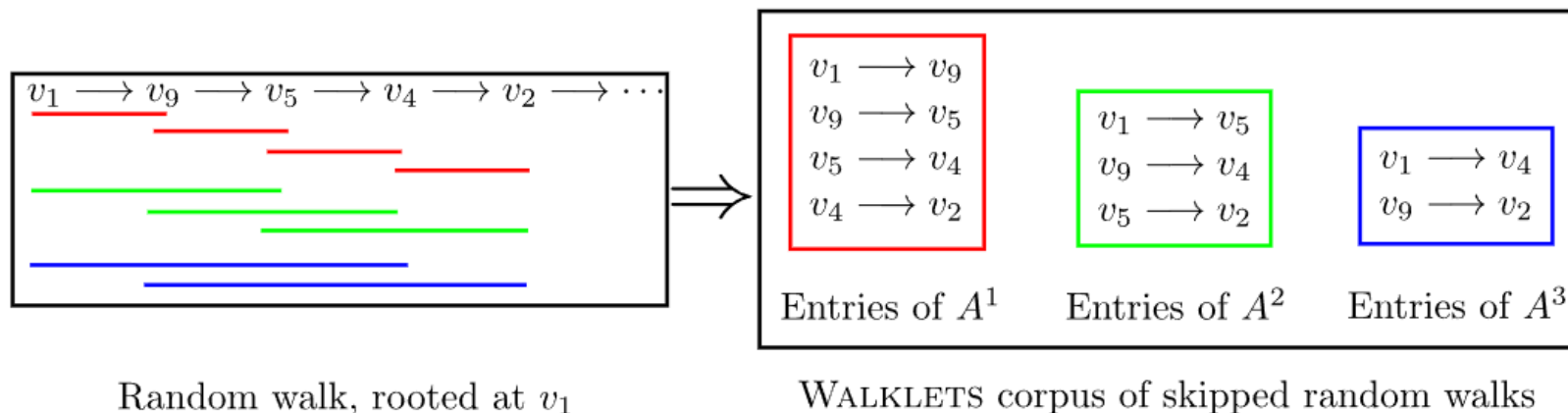


(b) WALKLETS Fine Representation



(c) WALKLETS Coarse Representation

- Generates truncated random walks on the graph, but "skips" over nodes at different scales
 - Results in multiple "corpora" of node co-occurrences at different scales
- Using skip-gram models to learn separate embeddings from each of these corpora
 - Learns multiple embeddings capturing relationships at different scales explicitly
- Optimization by Stochastic Gradient Descent



- Optimize embeddings to **maximize likelihood of random walk co-occurrences**

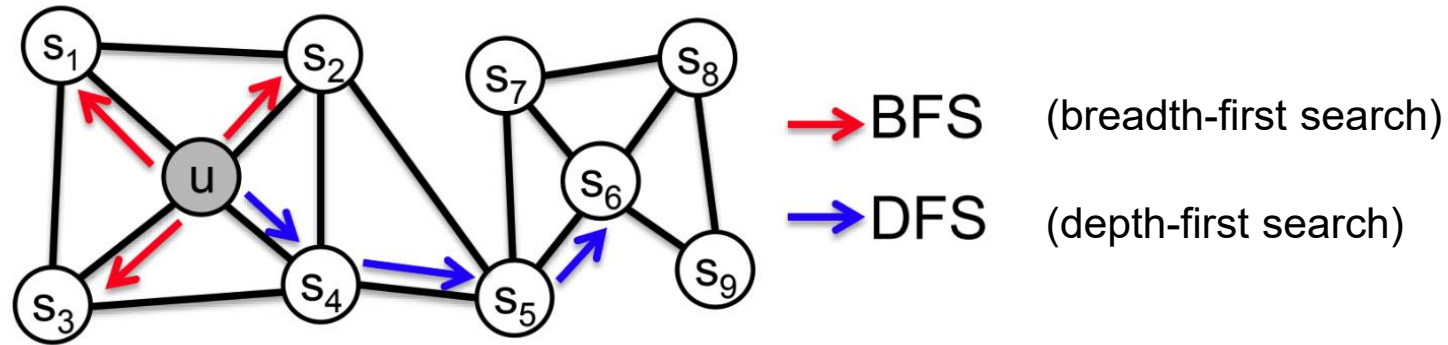
$$J = - \sum_{v_i, v_j \in C_k} \log \Pr(v_i | v_j)$$

- (Stochastic) Gradient Descent: a simple way to minimize J
- **SGD Algorithm:** evaluate it for each individual training example
 - Initialize z_u at some randomized value for all nodes u
 - Iterative until J converges:
 - Sample a node u , for all v calculate the derivative $\frac{\partial J^{(u)}}{\partial z_v}$
 - For all v , update:

$$z_v \leftarrow z_v - \eta \frac{\partial \mathcal{L}^{(u)}}{\partial z_v}$$

Learning rate

- **Idea:** Use flexible, **biased random walks** that can trade-off between local and global views of the network (Grover and Leskovec, 2016)
- Two classic strategies to define a neighbourhood $N_R(u)$ of a given node u

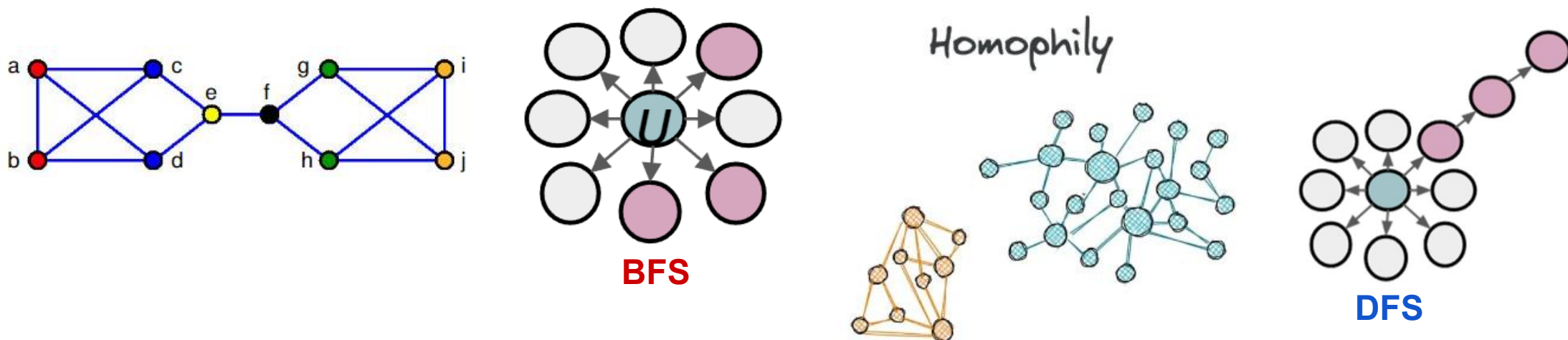


- Walk of length ($N_R(u)$ of size 3):

$$N_{BFS}(u) = \{s_1, s_2, s_3\} \quad \text{Local view}$$

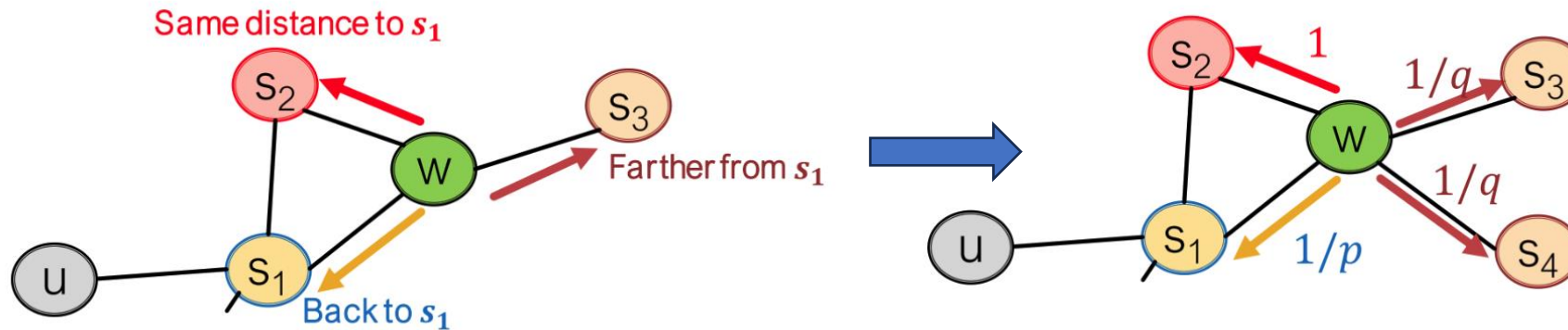
$$N_{DFS}(u) = \{s_4, s_5, s_6\} \quad \text{Global view}$$

- **Structural Equivalence:** if and only if they have the same structure
- **Homophily:** tendency of individuals to associate and bond with similar others
 - Neighborhoods sampled by **BFS** corresponding to **structural** equivalences
 - **DFS** explore macro-view of the network, which infers communities: **homophily** equivalence



- Biased fixed-length random walk R that given a node u generates neighborhood $N_R(u)$
 - Two parameters of random walks:
 - Return parameter p : Return to the previous node
 - In-out parameter q : Moving outwards (DFS) vs. inwards (BFS) from the previous node

- Biased 2nd-order random walks explore network neighbourhood
- Walker just traversed edge (s_1, w) and is at w , now he/she can go using a bias probability:



$1/p, 1, 1/q$ are unnormalized probs

p : return parameter

q : “walk away” parameter

- BFS-like walk: Low value of p
- DFS-like walk: Low value of q

$w \rightarrow$

Target t	Prob.	Dist. (s_1, t)
s_1	$1/p$	0
s_2	1	1
s_3	$1/q$	2
s_4	$1/q$	2

➤ Motivation:

- Preserving network proximities: LINE seeks to preserve both the **first-order proximity** (**direct connections between nodes**) and **second-order proximity** (**shared neighborhood structures**) in the learned embeddings
- Scalable objective functions:
 - Consider first-order and second-order proximities separately
 - Different network types: directed, undirected, weighted or unweighted
- An edge-sampling algorithm is proposed to help stochastic gradient descent on weighted edges

- First-order proximity: local pairwise proximity between two connected nodes
- For each node pair (v_i, v_j)
 - If $(v_i, v_j) \in E$, the first-order proximity between v_i and v_j is w_{ij}
 - Otherwise, the first-order proximity between v_i and v_j is 0
- Given an undirected edge (v_i, v_j) , the joint probability of v_i and v_j :

$$p_1(v_i, v_j) = \frac{1}{1 + \exp(-\vec{u}_i^T \cdot \vec{u}_j)}$$

$\vec{u}_i$: Embedding of node v_i

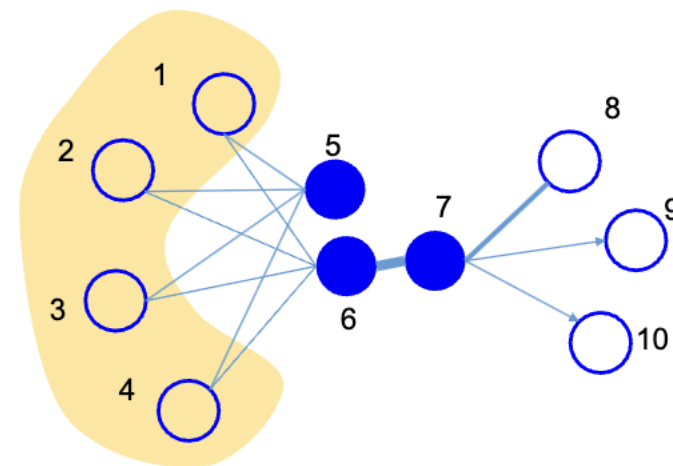
$$\hat{p}_1(v_i, v_j) = \frac{w_{ij}}{\sum_{(i', j')} w_{i' j'}}$$

- Objective:

$$O_1 = d(\hat{p}_1(\cdot, \cdot), p_1(\cdot, \cdot))$$

KL-divergence

$$\propto - \sum_{(i, j) \in E} w_{ij} \log p_1(v_i, v_j)$$

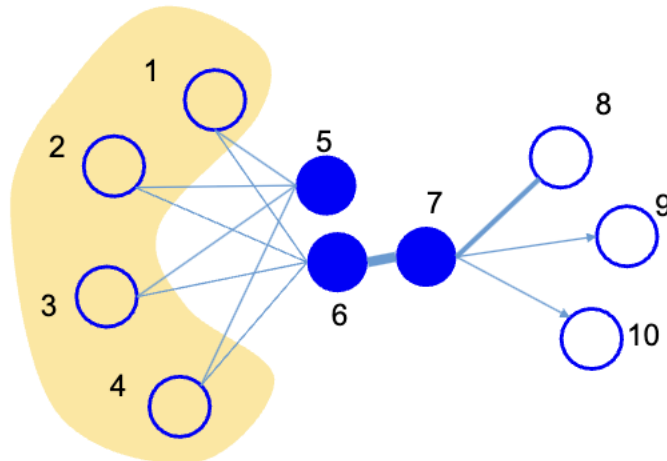


First-order: node 6 and 7

- Second-order proximity: captures the 2-step relations between each pair of nodes
- For each node pair (v_i, v_j)
 - Determine by the number of common neighbours shared by the two nodes

$$\hat{p}_u = (w_{u1}, w_{u2}, \dots, w_{u|V|})$$

$$\hat{p}_v = (w_{v1}, w_{v2}, \dots, w_{v|V|})$$



Node 5 and 6 have a large second-order proximity

$$\hat{p}_5 = (1, 1, 1, 1, 0, 0, 0, 0, 0, 0)$$

$$\hat{p}_6 = (1, 1, 1, 1, 0, 0, 1, 0, 0, 0)$$

- Given an undirected edge (v_i, v_j) , the joint probability of v_i and v_j :

$$p_2(v_j|v_i) = \frac{\exp(\vec{u}_j'^T \cdot \vec{u}_i)}{\sum_{k=1}^{|V|} \exp(\vec{u}_k'^T \cdot \vec{u}_i)},$$

$\vec{u}_i$: Embedding of node v_i when i is a source node.

$\vec{u}_i'$: Embedding of node v_i when i is a target node.

$$\hat{p}_2(v_j|v_i) = \frac{w_{ij}}{\sum_{k \in V} w_{ik}}$$

- Objective:

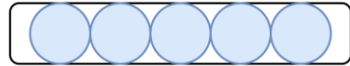
λ_i : Prestige of node in the network $\lambda_i = \sum_j w_{ij}$

$$O_2 = \sum_{i \in V} \lambda_i d(\hat{p}_2(\cdot|v_i), p_2(\cdot|v_i)),$$

$$\propto - \sum_{(i,j) \in E} w_{ij} \log p_2(v_j|v_i).$$

- Concatenate the embeddings individually learned by the two proximity:

First-order:



Second-order:



- Stochastic Gradient Descent + Negative Sampling
 - Randomly sample an edge and multiple negative edges
- The gradient w.r.t the embedding with edge (i,j)

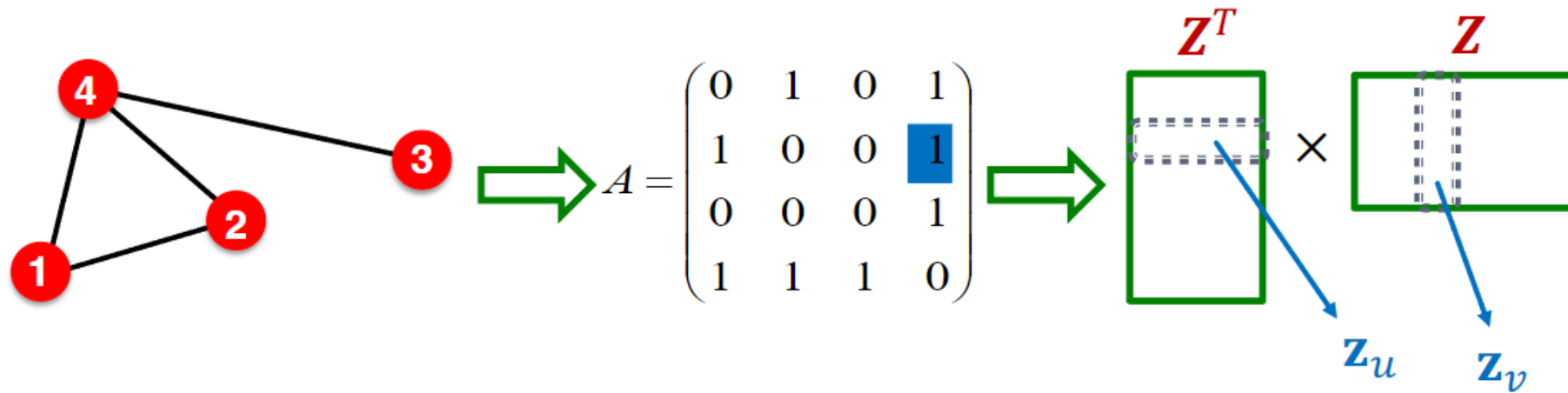
$$\frac{\partial O_2}{\partial \vec{u}_i} = w_{ij} \cdot \frac{\partial \log p_2(v_j | v_i)}{\partial \vec{u}_i}$$

Multiplied by the weight of the edge w_{ij}

- Problematic when the weights of the edges diverge
 - The scale of the gradients with different edges diverges
- **Solution:** Edge sampling
 - Sample the edges according to their weights and treat the edges as binary
- Complexity: $\Theta(dK|E|)$
 - Linear to the dimension d , number of negative samples K , and number of edges E

- One of the simplest and most intuitive approaches to defining similarity:
 - Adjacency between two nodes v and u
- Similarity: Two nodes are adjacent to one another within the structure of the graph

$$\mathbf{z}_v^T \mathbf{z}_u = A_{u,v} \text{ or } \mathbf{Z}^T \mathbf{Z} = A$$



- Encoding: Find the embedding matrix $\mathbf{Z}$ that minimizes the loss function $\mathcal{L}$

$$\mathcal{L} = \sum_{(u,v) \in V \times V} \| \mathbf{z}_u^\top \mathbf{z}_v - \mathbf{A}_{u,v} \|^2$$

Diagram illustrating the loss function $\mathcal{L}$ for graph embeddings:

- $\mathcal{L}$: loss (what we want to minimize)
- $\sum_{(u,v) \in V \times V}$: sum over all node pairs
- $\mathbf{z}_u^\top \mathbf{z}_v$: embedding similarity
- $\mathbf{A}_{u,v}$: (weighted) adjacency matrix for the graph

- Solution:
 - Option 1: Use stochastic gradient descent (SGD) as a general optimization method
 - Option 2: Solve matrix decomposition solvers (e.g., SVD or QR decomposition routines)
 - **Matrix Factorization-based**

- Let A be an $m \times n$ adjacency matrix. The factorization of A takes the form

$$A = USV^T$$

where U is a $m \times m$ orthogonal matrix, V^T is a $n \times n$ orthogonal matrix and S is a $m \times n$ diagonal matrix

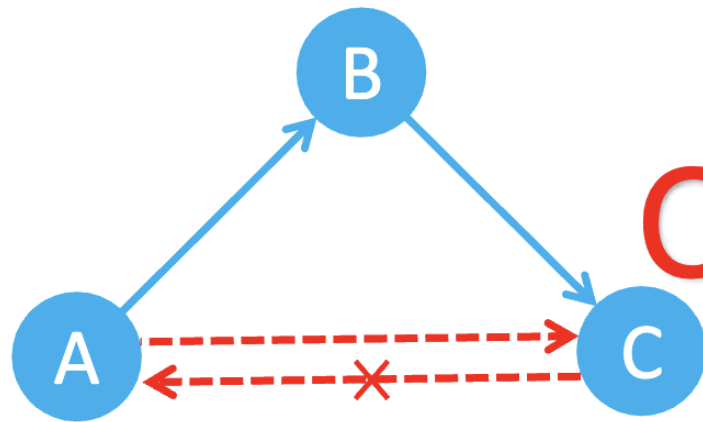
- Factorizing to low-rank approximations based on minimizing the sum-squared distance using **Singular Value Decomposition**
- To decompose:
 - Evaluate the n eigenvectors v_i and eigenvalues λ_i of $A^T A$
 - Make a matrix V from the normalized vectors v_i
 - Make a diagonal matrix S from the square roots of the eigenvalues

$$\sigma_i = \sqrt{\lambda_i} \quad \text{and} \quad \sigma_1 \geq \sigma_2 \geq \sigma_3 \dots$$

- Find U : $A = USV^T \Rightarrow US = AV \Rightarrow U = AVS^{-1}$

- **Ideas:** Critical property in directed graph - **Asymmetric Transitivity**
- Transitivity is **Asymmetric** in directed graph
- **Challenge:** incorporate asymmetric transitivity in graph embedding
- **Problem:** metric space is **symmetric**

$$\mathbf{z}_v^T \mathbf{z}_u = A_{u,v} \text{ or } \mathbf{Z}^T \mathbf{Z} = A$$



Asymmetric transitivity

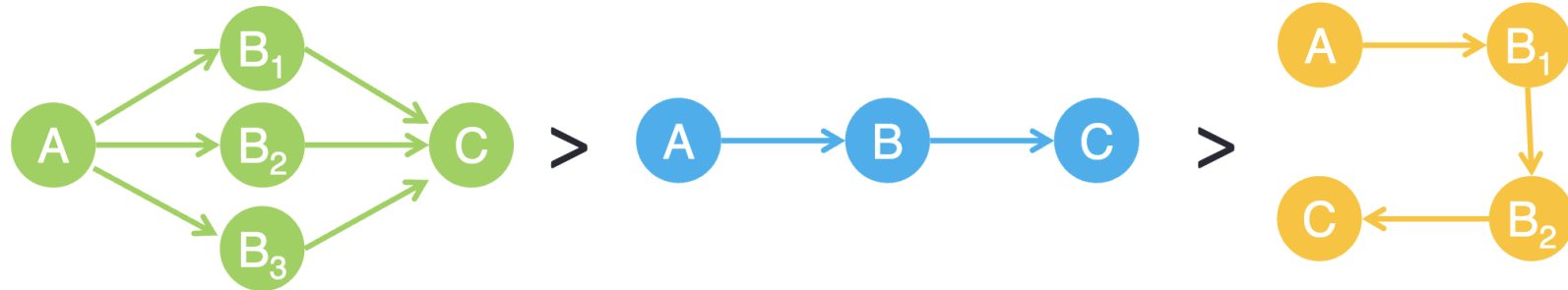
Conflict !



Matrix space

➤ High-order Proximity with asymmetric transitivity:

- Asymmetry: not symmetric in directed graph
- Transitivity:
 - More directed paths, larger similarity
 - Shorter paths, larger similarity



➤ Compare A → C similarity: Katz Index

$$\mathbf{s}^{Katz} = \sum_{l=1}^{+\infty} (\beta \cdot A)^l$$

➤ Preserve high-order proximity embedding:

➤ Katz Index modified:

$$\mathbf{S}^{Katz} = \sum_{l=1}^{+\infty} (\beta \cdot A)^l = (I - \beta \cdot A)^{-1} \cdot (\beta \cdot A)$$

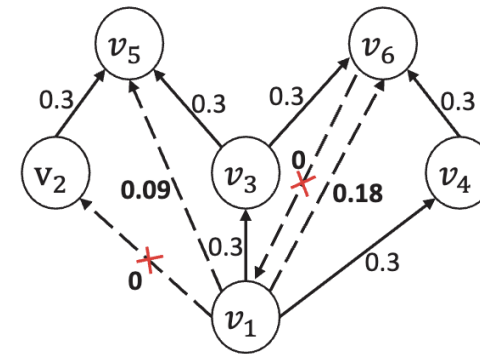
➤ Objective:

$$\min_{U_s, U_t} \|\mathbf{S} - U_s \cdot U_t^T\|_F^2$$

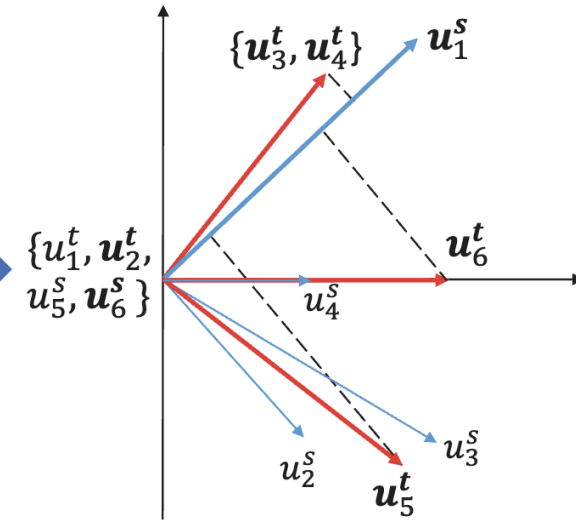
$$\mathbf{S} = M_g^{-1} \cdot M_l$$

Embedding similarity

where M_g, M_l are polynomial of adjacency matrix or proximity measurements: Katz, Adamic Adar, PageRank, Common Neighbours



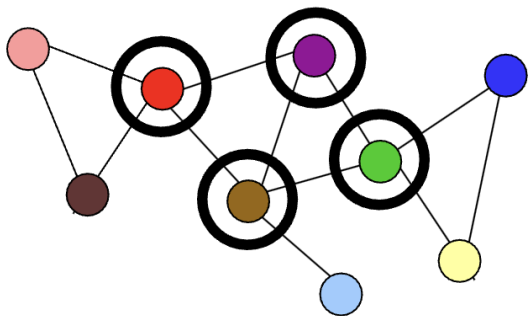
HOPE



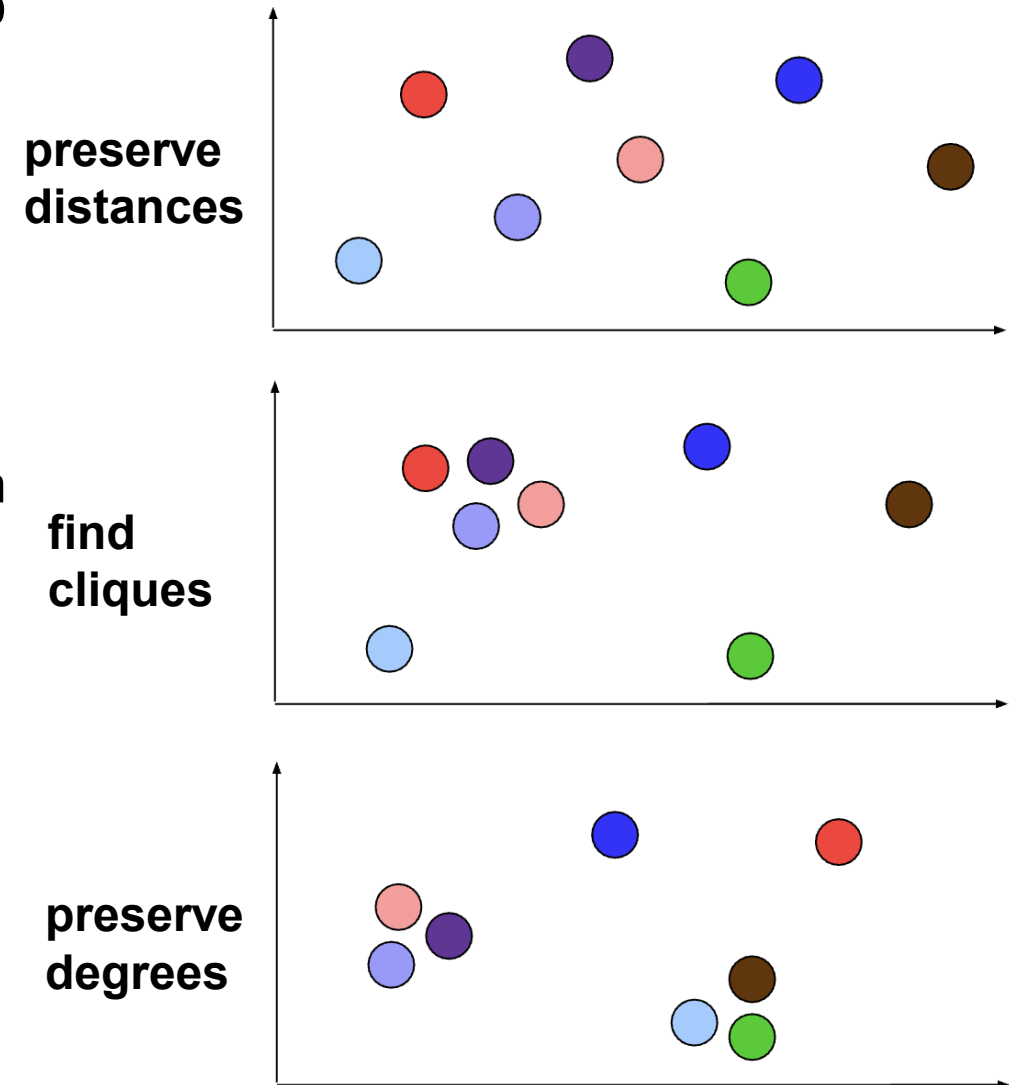
➤ Solving by **Generalized Singular Value Decomposition**: decompose $\mathbf{S}$ without calculating it

➤ **Linear complexity** w.r.t. the volume of data (i.e. edge number)

- Network embedding: map network nodes into Euclidean space
- Structural Identity:
 - Nodes in networks have specific roles
 - E.g., individuals, web pages, proteins, etc
 - Structural identity: identification of nodes based on network structure (no other attribute)
 - often related to role played by node
 - **Automorphism**: strong structural equivalence



Red, Green: automorphism
Purple, Brown: structurally similar



- **Ideas:** Based on structural identity
- Structural similarity does not depend on hop distance
 - Neighbour nodes can be different, far away nodes can be similar
- Structural identity as a hierarchical concept
 - Depth of similarity varies
- Flexible four step procedure
 - Operational aspect of steps are flexible

➤ $g(D_1, D_2)$: distance between two ordered sequences

- Cost of pairwise alignment: $\max(a, b) / \min(a, b) - 1$
- Optimal alignment by Dynamic Time Warping (DTW) in our framework

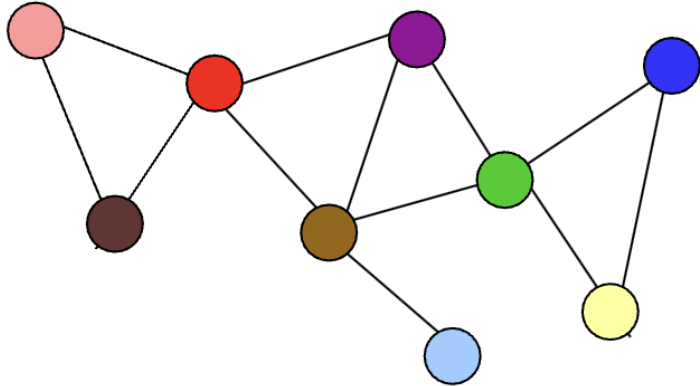
$s(R_0(u)) = 4$	$s(R_1(u)) = 1, 3, 4, 4$	$s(R_2(u)) = 2, 2, 2, 2$
$s(R_0(v)) = 3$	$s(R_1(v)) = 4, 4, 4$	$s(R_2(v)) = 1, 2, 2, 2, 2$
$g(. , .) = 0.33$	$g(. , .) = 3.33$	$g(. , .) = 1$

➤ $f_k(u, v)$: structural distance between nodes u and v considering first k rings

➤ $f_k(u, v) = f_{k-1}(u, v) + g(s(R_k(u)), s(R_k(v)))$

$f_0(u, v) = 0.33$	$f_1(u, v) = 3.66$	$f_2(u, v) = 4.66$
--------------------	--------------------	--------------------

- Encodes structural similarity between all node pairs



$$w(u_k, u_{k+1}) = \log(\Gamma_k(u) + e), \quad k = 0, \dots, k^* - 1$$

$$w(u_k, u_{k-1}) = 1, \quad k = 1, \dots, k^*$$

- Each layer is weighted complete graph

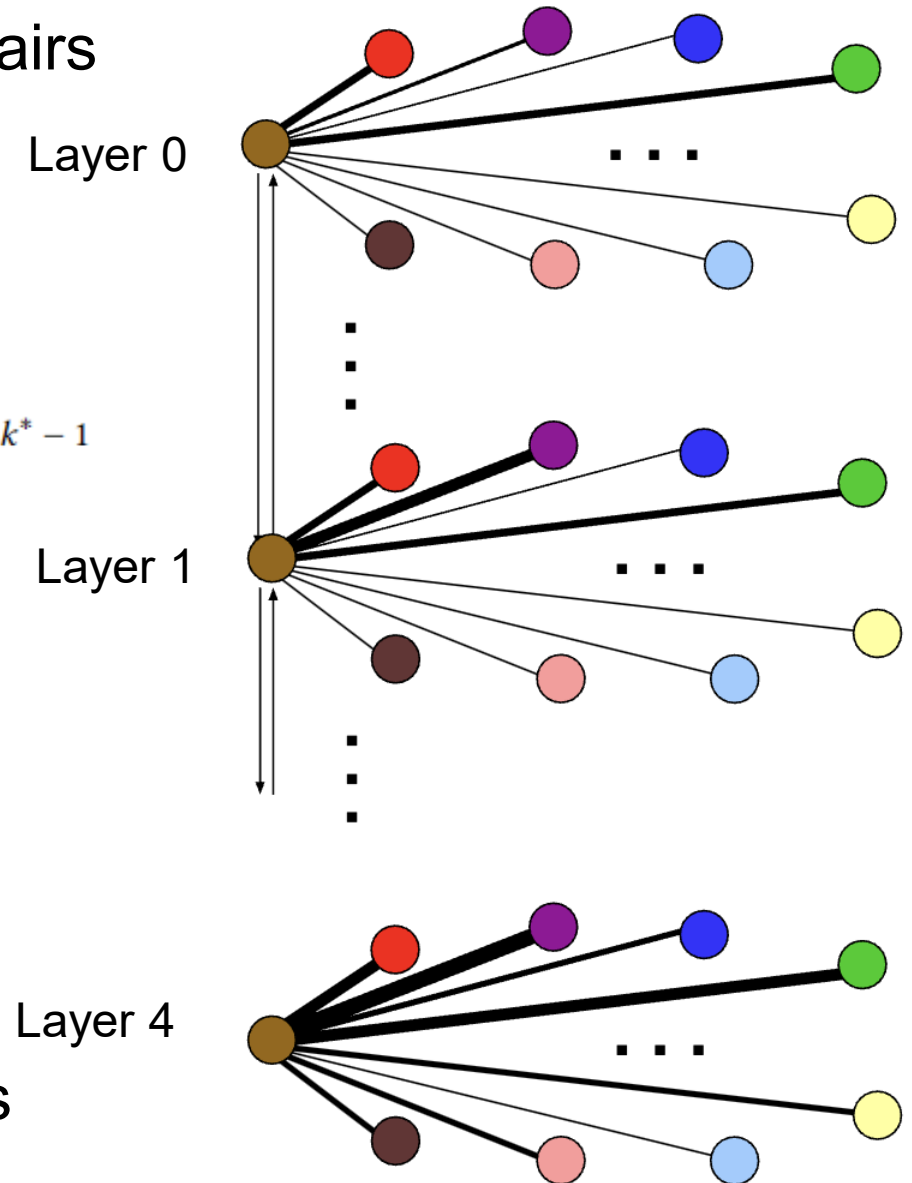
- Corresponds to similarity hierarchies

$$\Gamma_k(u) = \sum_{v \in V} \mathbb{1}(w_k(u, v) > \overline{w_k})$$

- Edge weights in layer k

$$w_k(u, v) = e^{-f_k(u, v)}, \quad k = 0, \dots, k^*$$

- Connect corresponding nodes in adjacent layers



- Context generated by biased random walk (same as Node2vec)
 - Walking on multi-layer graph
- Walk in current layer with probability p : $p_k(u, v) = \frac{e^{-f_k(u, v)}}{Z_k(u)}$
 - Choose neighbour according to edge weight where $Z_k(u)$ is the normalization factor for vertex u in layer k ,

$$Z_k(u) = \sum_{\substack{v \in V \\ v \neq u}} e^{-f_k(u, v)}$$
 - RW prefers more similar nodes
- Change layer with probability $1-p$
 - Choose up/down according to edge weight
 - RW prefer layer with less similar neighbours

$$p_k(u_k, u_{k+1}) = \frac{w(u_k, u_{k+1})}{w(u_k, u_{k+1}) + w(u_k, u_{k-1})}$$

$$p_k(u_k, u_{k-1}) = 1 - p_k(u_k, u_{k+1})$$

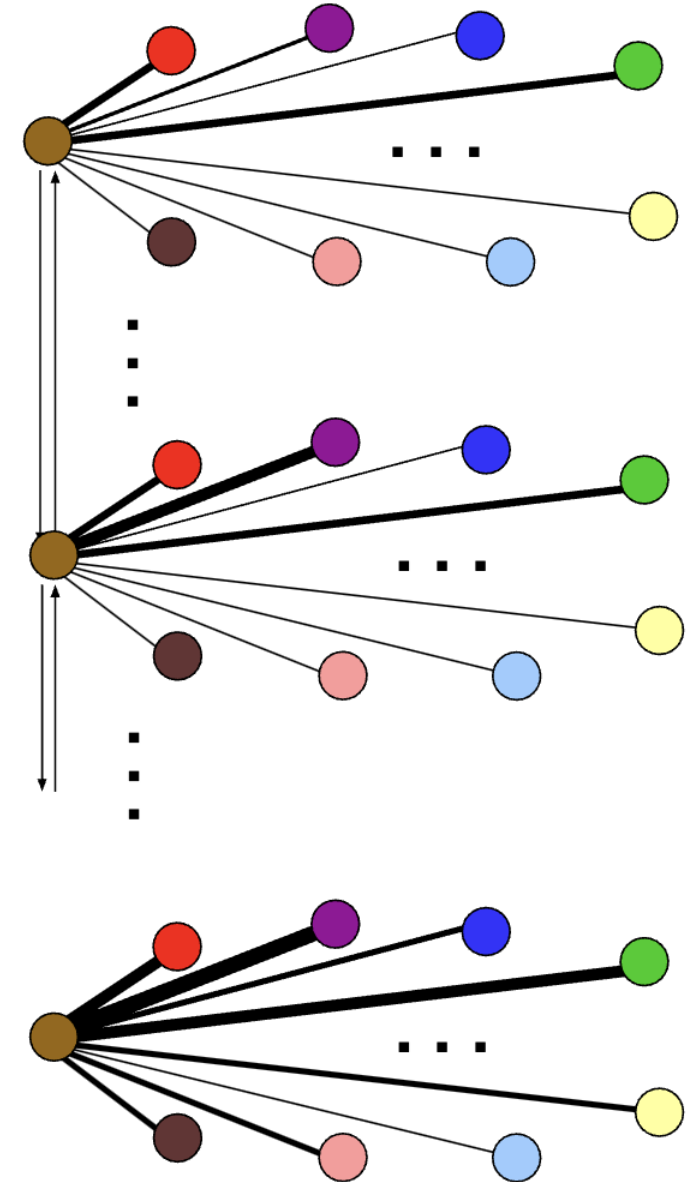
- For each node, generate set of independent and relative short random walks

- Context for node; sentences of a language



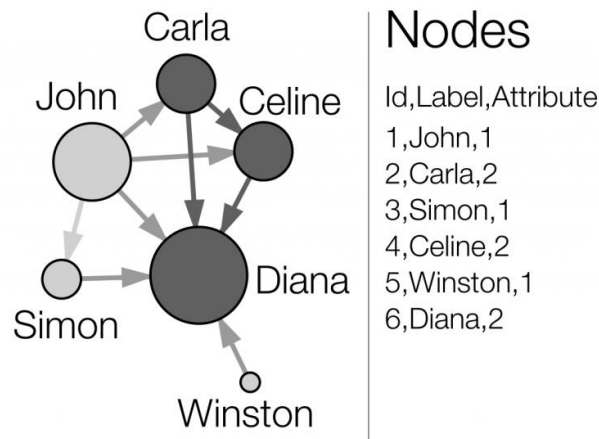
- Train a neural network to learn latent representation for nodes

- maximize probability of nodes within context
 - Skip-gram (Hierarchical Softmax) adopted

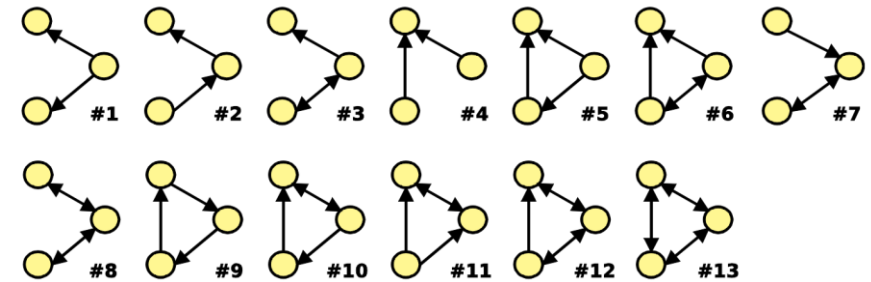


- Reduce time to generate/store multi-layer graph and context for nodes:
 - Option 1: Reduce length of degree sequences
 - Use pairs (degree, number of occurrences)
 - Option 2: Reduce number of edges in multi-layer graph
 - Only log n neighbours per node
 - Option 3: Reduce number of layers in multi-layer graph
 - Fixed (small) number of layers
 - Scales quasi-linearly
 - Over 1 million nodes

- **Ideas:** struc2vec and conventional methods (DeepWalk, Node2vec) learn in node representation
 - Role2Vec learns embeddings for "node types/roles" determined by node attributes/features
- **Solution:** "attributed random walks"
 - Maps nodes to node types/roles using node attributes like **motif counts, degrees** etc
 - Generates attributed random walks over the node types instead of node IDs
 - Finally, it learns embeddings for the node types rather than individual nodes



- Given a network, most of the time, some subgraphs are “overrepresented”
- A connected graph that has many occurrences in a network is called a **motif** of the network
- Assume set of occurrences G' in G is $occ_G(H)$
 - Cardinality of $occ_G(H)$ in G is **frequent**
 - How to know if G' is **frequent** in G ?



➡ Compute the probability that $occ_N(G') \geq occ_G(G')$ for a random network N

G' is said to be frequent in G if this probability is small enough

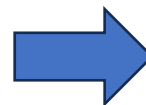
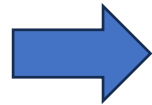
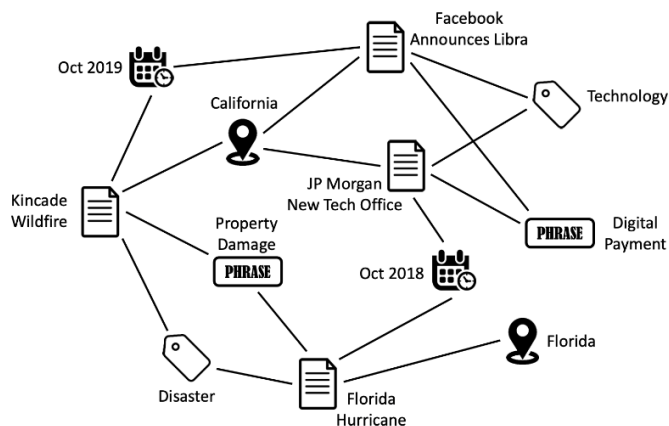
To compute this probability, we need to have a distribution over networks

- **Input:** Graph G , node attribute matrix X , embedding dimension D , number of walks per node R , walk length L , context window size ω
- If X is not available, extract structural features like motif counts from the graph structure itself and use those as node attributes in X
- Apply logarithmic binning to the node attribute values in X . Map nodes to node types/roles using a function $\phi(x)$ that takes the node attribute vector x as input
 - Two types of ϕ functions:
 - a) Simple functions like concatenation of attribute values
 - b) Low-rank matrix factorization of X
- Precompute random walk transition probabilities π . Generate R attributed random walks of length L for each node, using the node type mapping ϕ instead of node IDs
- Learn embeddings using stochastic gradient descent on the attributed random walks, optimizing the probability of observing the context node types in the walks
- **Output:** Learned embeddings for each node type $w \in W$, where W is the set of node types found by ϕ

- Mostly same as Node2Vec, except their work are considered in terms of node attribute, not node representation
- There are many advantage from node attributed embedding:
 - Embeddings generalize to new nodes/graphs (inductive learning)
 - Better captures structural node roles
 - Space-efficient as it learns fewer embeddings for node types instead of all nodes
 - Supports attributed graphs

From Homogeneous to Heterogeneous Network Embedding: Problem 84

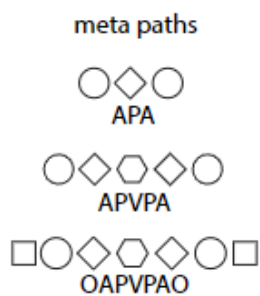
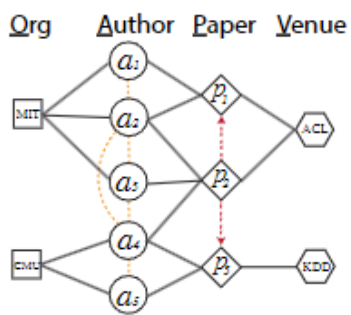
- Previous works are considering **homogeneous network**
 - Only **one type** of **nodes** and **edges**
 - Real world is ubiquitous. For example: social media websites like Facebook contain a set of node types, such as users, posts, groups and tags
 - Special case of heterogeneous network
- Input: A heterogeneous information network $G = (V, E, T)$. T is object relation type
- Output: $X \in R^{|V| \times d}$, $d \ll |V|$, d -dim vector X_v for each node V



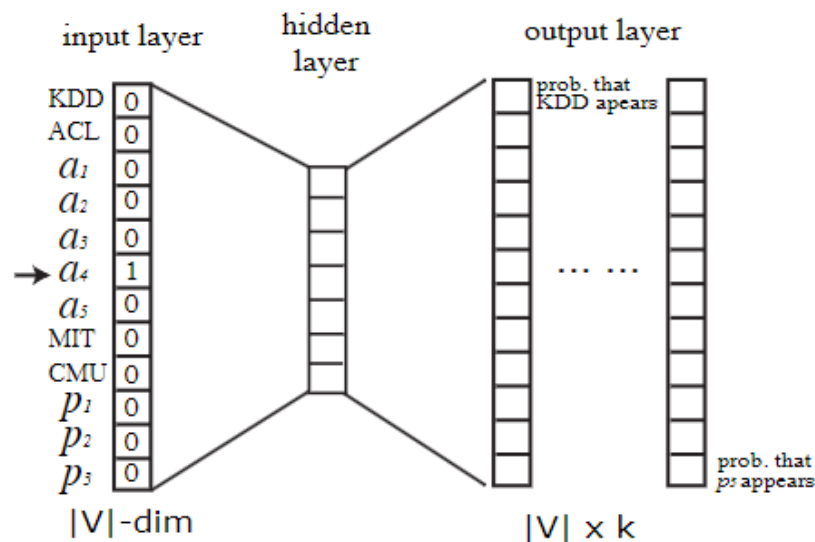
Latent representation vector

- How do we effectively preserve the concept of “node-context” among multiple types of nodes?
 - E.g., users, posts, groups and, tags in social heterogeneous networks
 - Or authors, papers, & venues in academic heterogeneous networks
- Can we directly apply homogeneous network embedding architectures to heterogeneous networks?
- It is also difficult for conventional meta-path-based methods to model similarities between nodes without connected meta-paths

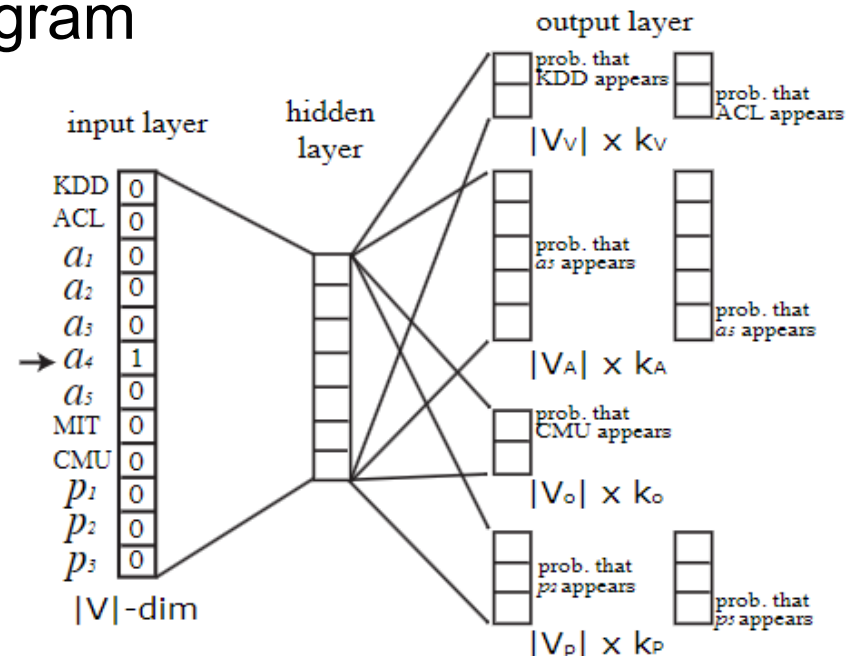
- **Solution:** meta-path based random walk (inspired by DeepWalk and Node2Vec)
 - metapath2vec and metapath2vec++
- **Goal:** to generate paths that can capture both the **semantic** and **structural correlations** between different types of nodes, facilitating the transformation of heterogeneous network structures into skip-gram



Heterogenous network



Skip-gram in metapath2vec

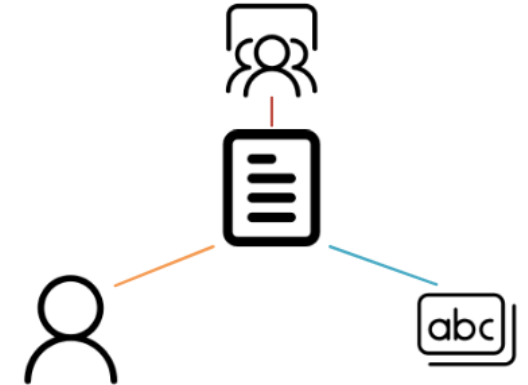


Skip-gram in metapath2vec++

- Network schema $S = (V, R)$ of graph G
 - Directed graph defined over node types V and with edges as relations from R

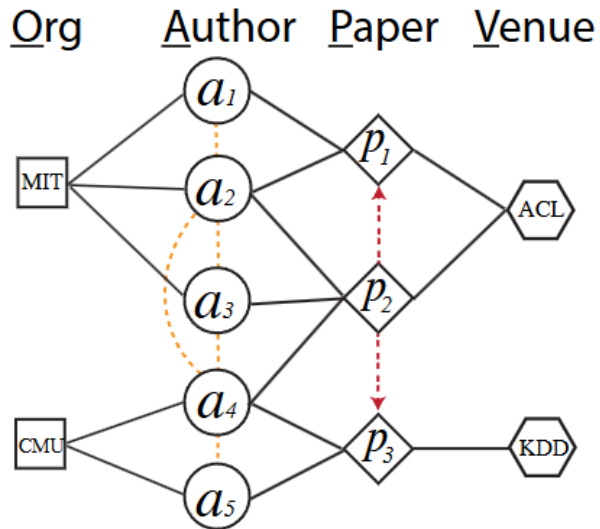
- meta-path: based on network schema S

- Denoted as $V_1 \xrightarrow{R_1} V_2 \xrightarrow{R_2} \dots V_t \xrightarrow{R_t} V_{t+1} \dots \xrightarrow{R_{l-1}} V_l$
- Node types $V_1, V_2, \dots, V_l \in V$ and edge type $R_1, R_2, \dots, R_{l-1} \in R$



- Each meta-path captures the proximity between the nodes on its two ends from a particular semantic perspective

➤ Metapath2Vec:



- Given a meta-path scheme

$$\mathcal{P}: V_1 \xrightarrow{R_1} V_2 \xrightarrow{R_2} \dots V_t \xrightarrow{R_t} V_{t+1} \dots \xrightarrow{R_{l-1}}$$

- The transition probability at step i is defined as

$$p(v^{i+1}|v_t^i, \mathcal{P}) = \begin{cases} \frac{1}{|N_{t+1}(v_t^i)|} & (v^{i+1}, v_t^i) \in E, \phi(v^{i+1}) = t+1 \\ 0 & (v^{i+1}, v_t^i) \in E, \phi(v^{i+1}) \neq t+1 \\ 0 & (v^{i+1}, v_t^i) \notin E \end{cases}$$

- Recursive guidance for random walkers, i.e.,

$$p(v^{i+1}|v_t^i) = p(v^{i+1}|v_1^i), \text{ if } t = l$$

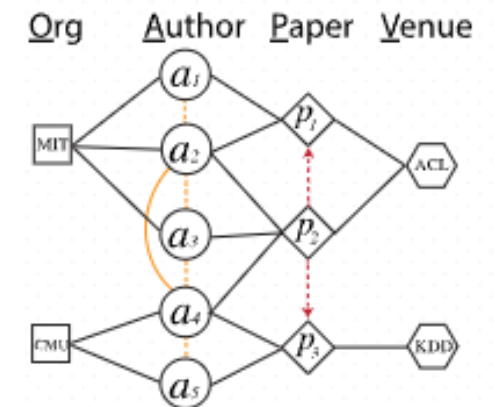
➤ **Metapath2Vec:**

➤ Given a meta-path scheme (Example):

OAPVPAO

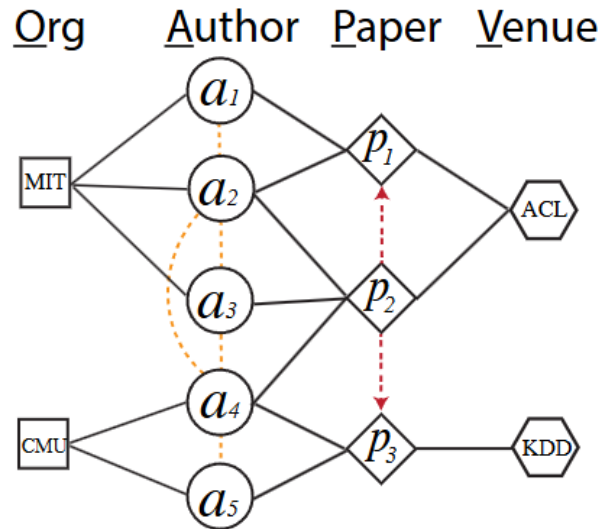
➤ In a traditional random walk procedure, the next step of a walker on node a_4 transitioned from node CMU can be all types of nodes surrounding it - a_2, a_3, a_5, p_2, p_3 , and CMU

➤ Under the meta-path scheme 'OAPVPAO', for example, the walker is biased towards paper nodes (P) given its previous step on an organization node CMU (O), following the semantics of this meta-path

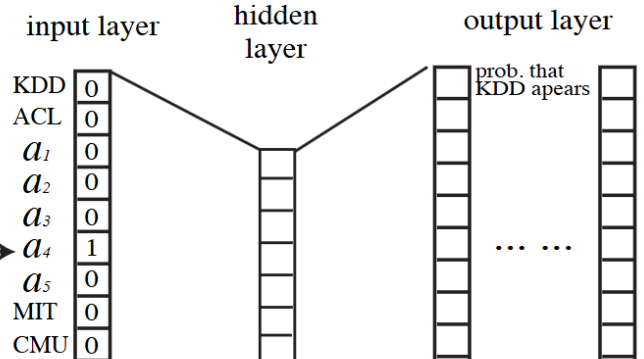
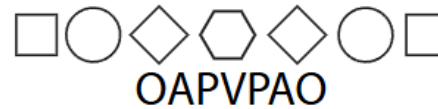
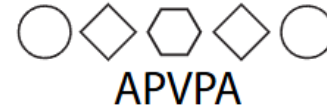
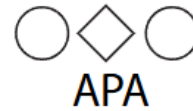


meta-path-based
random walks

skip-gram



meta paths



The potential issue of skip-gram for heterogeneous network embedding:

To predict the context node c_t (type t) given a node v , *metapath2vec* encourages all types of nodes to appear in this context position

➤ Softmax in Metapath2Vec

$$p(c_t|v; \theta) = \frac{e^{X_{c_t} \cdot X_v}}{\sum_{u \in V} e^{X_u \cdot X_v}}$$

Not consider node type

- **Metapath2Vec++**: Heterogeneous Skip-Gram
- **Objective function** (heterogeneous negative sampling):

$$O(X) = \log \sigma(X_{c_t} \cdot X_v) + \sum_{m=1}^M \mathbb{E}_{u_t^m \sim \mathbb{P}_t(u_t)} [\log \sigma(-X_{u_t^m} \cdot X_v)]$$

- **Softmax in Metapath2Vec++**

$$p(c_t|v; \theta) = \frac{e^{X_{c_t} \cdot X_v}}{\sum_{u_t \in V_t} e^{X_{u_t} \cdot X_v}}$$

Consider node type t

- **Stochastic Gradient Descent Updates**

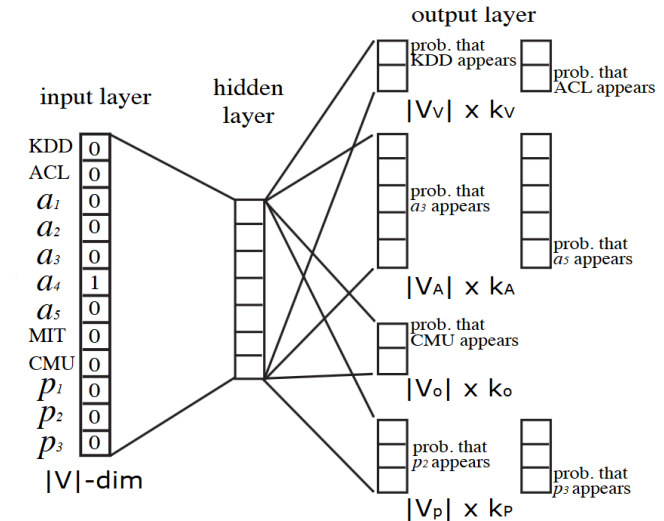
- For the negative sample u_m^t (where $m = 0$ denotes the positive context c_t):

$$\frac{\partial O(X)}{\partial X_{u_m^t}} = (\sigma(X_{u_m^t} \cdot X_v) - I_{c_t}[u_m^t])X_v$$

- For the center node v :

$$\frac{\partial O(X)}{\partial X_v} = \sum_{m=0}^M (\sigma(X_{u_m^t} \cdot X_v) - I_{c_t}[u_m^t])X_{u_m^t}$$

where $I_{c_t}[u_m^t]$ is an indicator function that equals 1 if $u_m^t = c_t$ (the actual context node), and 0 otherwise.



Representation Learning on Heterogeneous Information Networks (HINs)

What is a HIN?

A directed graph

$$G = (V, E, \Phi, \Psi)$$

where $|A| > 1$ or $|R| > 1$ - i.e., nodes and edges belong to multiple types.

$|A|$ is the **number of unique node types** in the network.

- A is the set of all possible node types (e.g., $\{Author, Paper, Venue\}$).
- Φ is the function that maps each node v to its type in A .

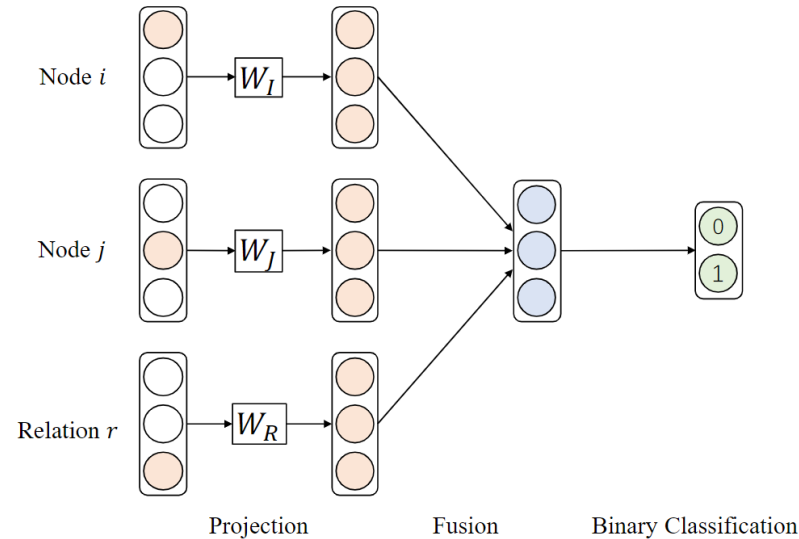
$|R|$ is the **number of unique edge (relationship) types** in the network.

- R is the set of all possible edge types (e.g., $\{Writes, Published_In\}$).
- Ψ is the function that maps each edge e to its type in R .

Why the condition $|A| > 1$ or $|R| > 1$ matters:

- If $|A| = 1$ and $|R| = 1$, the network is **homogeneous** (all nodes are the same type, all edges are the same type, like a simple friendship graph).
- A network is considered **heterogeneous (an HIN)** only if there is more than one type of node **OR** more than one type of relationship.

HIN2vec architecture



Core Idea: Reduce multi-relationship prediction into binary classification per relationship

Architecture:

- **Input:** three one-hot vectors $\vec{x}, \vec{y}, \vec{r}$ (two nodes + one meta-path)
- **Latent layer:** $W'_X \vec{x} \odot W'_Y \vec{y} \odot f_{01}(W'_R \vec{r})$ via *Hadamard product*
- **Output:** $\hat{P}(r | x, y) = \sigma(\sum W'_X \vec{x} \odot W'_Y \vec{y} \odot f_{01}(W'_R \vec{r}))$

Training Data Preparation

Phase	Description
Phase 1	Random walk-based training data sampling
Phase 2	Joint representation learning via HIN2Vec NN

Random Walk Sampling:

- For walk length l and max hop w : time complexity $O(l + w(l - w))$, space $O(|V| + l)$
- Scalable: subsequences of a walk reused as training samples for multiple meta-paths

Critical Issues Addressed:

1. **Cycles in random walks:** a path A_1, P_3, A_1 generates $\langle A_1, A_1, A-P-A \rangle$, a self-coauthorship, which is semantically invalid $\rightarrow$ eliminated by duplicate node checking
2. **Negative sampling node type:** replacing nodes with arbitrary types introduces erroneous negatives $\rightarrow$ same-type replacement enforced
3. **Regularization function:** Sigmoid keeps W_R values in $(0,1)$, amplifying node vector noise; Binary Step is discrete and more stable $\rightarrow$ Binary Step preferred

Objective Function:

The objective function is designed to train the HIN2Vec model as a binary classifier. It determines how well the model predicts whether a specific relationship r exists between two nodes, x and y . The objective is to **maximize** the log-likelihood $\log O$ across the entire training set D :

$$\log O = \sum_{x,y,r \in D} L(x,y,r) \log P(r|x,y) + [(1 - L(x,y,r)) \log(1 - P(r|x,y))]$$

- **Ground Truth ($L(x,y,r)$):** This is a binary indicator from the training data
 - If the relationship r (e.g., *Author-writes-Paper*) exists between x and y , $L = 1$.
 - If the relationship does not exist (generated via negative sampling), $L = 0$.
- **Predicted Probability ($P(r|x,y)$):** This is the model's output, calculated using the latent vectors of the nodes and the relationship:

$$P(r|x,y) = \text{sigmoid} \left(\sum W'_X \vec{x} \odot W'_Y \vec{y} \odot f_{01}(W'_R \vec{r}) \right)$$

The model uses a Hadamard product (element-wise multiplication $\odot$) of the three vectors, sums the result, and applies a sigmoid function to squash the value between 0 and 1.

Objective Function:

The objective function is designed to train the HIN2Vec model as a binary classifier. It determines how well the model predicts whether a specific relationship r exists between two nodes, x and y . The objective is to **maximize** the log-likelihood $\log O$ across the entire training set D :

$$\log O = \sum_{x,y,r \in D} \boxed{L(x,y,r) \log P(r|x,y)} + \boxed{[(1 - L(x,y,r)) \log(1 - P(r|x,y))]}$$

Maximizing Positive Samples: When $L = 1$, the second half of the equation becomes 0. The model focuses on making $P(r|x,y)$ as close to 1 as possible. This forces the node vectors for x and y and the relationship vector for r to be similar in the latent space.

Minimizing Negative Samples: When $L = 0$, the first half becomes 0. The model focuses on making $1 - P(r|x,y)$ as large as possible (meaning $P(r|x,y)$ becomes close to 0). This ensures that nodes without that specific relationship are pushed apart in the latent space.

Optimization via Backpropagation:

The model uses stochastic gradient descent (SGD) to update three sets of weights simultaneously:

- W_X : The embedding matrix for the "source" nodes.
- W_Y : The embedding matrix for the "target" nodes (often regularized to be the same as W_X).
- W_R : The embedding matrix for the meta-paths, allowing the model to learn the semantic meaning of different types of relationships.

Although many heterogeneous graph (HG) representation learning methods rely on meta-paths to model semantic proximity, meta-paths have several important limitations:

- **Manual Design Requirement**

Meta-paths mainly capture sequential relations and often fail to represent higher-order structural patterns such as **motifs** and complex subgraph interactions.

- **Limited Expressiveness**

Meta-paths mainly capture sequential relations and often fail to represent higher-order structural patterns such as motifs and complex subgraph interactions.

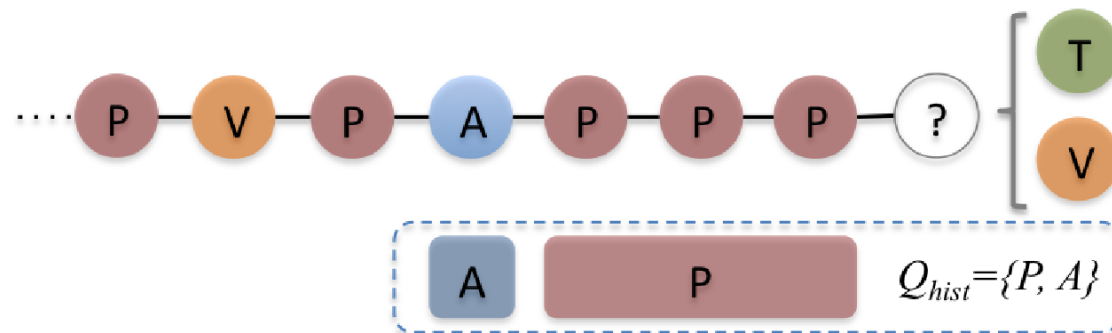
- **Discrete Connectivity Constraint**

Meta-path-based methods operate in a discrete relational space. If two nodes are not explicitly connected through predefined structural paths, their latent semantic relationships cannot be effectively captured.

- **Scalability Challenges**

The number of possible meta-paths grows rapidly as schema complexity increases, making path selection and computation expensive.

- **Solution:** propose **JUST**, which performs random walks by probabilistically deciding whether to "**jump**" to a different node type or "**stay**" in the same node type
 - No reliance on predefined meta-paths
 - **Balance** between **homogeneous** edges (same node type) and **heterogeneous** edges (across node types)
 - Balances the node distribution over different node types by controlling the jumping behavior



- Perform random walks on the heterogeneous graph, but probabilistically decide at each step
 - **Jump** to a target domain q : uniformly sampling one node from those in a target domain q connected to v_i via heterogeneous edge

$$V_{jump}^q(v_i) = \{v | (v_i, v) \in E_{he} \vee (v_i, v) \in E_{he}, \phi(v) = q\}$$

- **Stay** in the current domain: uniformly sampling one node from those connected to v_i via homogeneous edges

$$V_{stay}(v_i) = \{v | (v_i, v) \in E_{ho} \vee (v_i, v) \in E_{ho}\}$$

➤ Decision 1: Jump or Stay

$$Pr_{stay}(v_i) = \begin{cases} 0, & \text{if } V_{stay}(v_i) = \emptyset \\ 1, & \text{if } \{V_{jump}^q(v_i) | q \in Q, q \neq \phi(v_i)\} = \emptyset \\ \alpha^l, & \text{otherwise} \end{cases}$$

if no heterogeneous neighbors



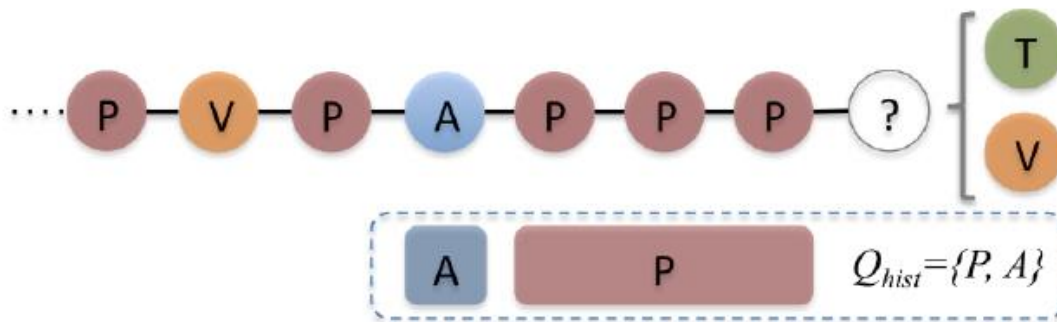
Where:

- $\alpha \in [0, 1]$: initial stay probability
- l : consecutive steps in the same domain
- **Exponential decay** penalizes prolonged stays in one domain
- Smaller $\alpha \rightarrow$ more heterogeneous edges (more jumps)

➤ Decision 2: Where to Jump?

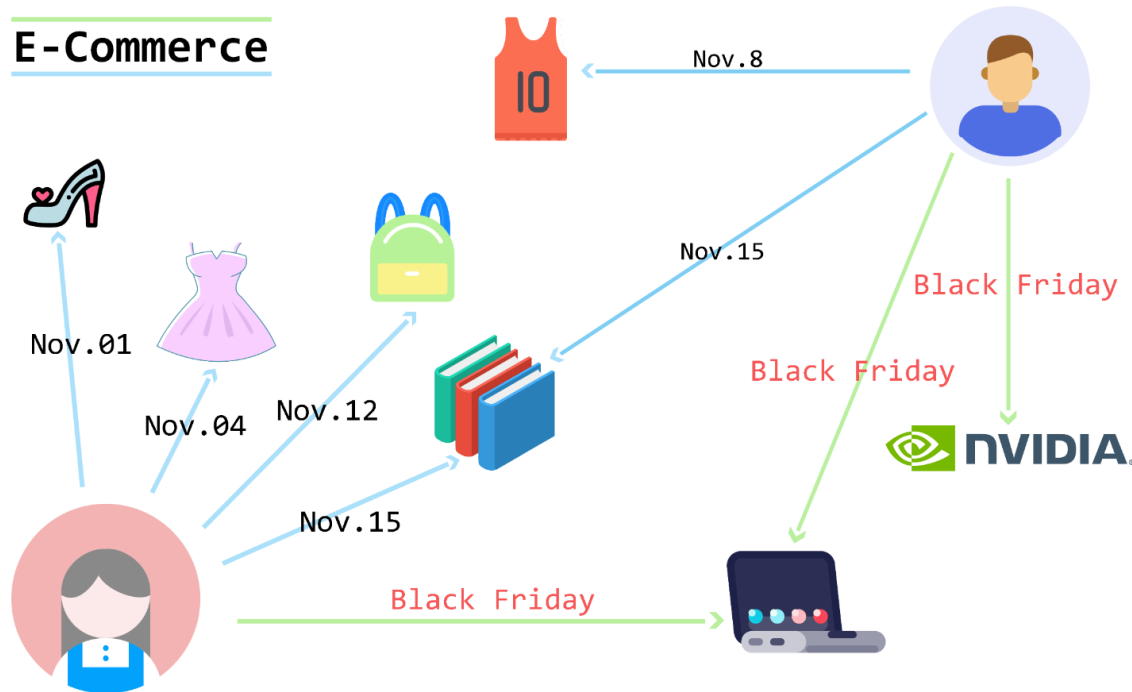
Define Q_{hist} : fixed-length queue of last m visited domains

$$Q_{\text{Jump}}(v_i) = \begin{cases} \{q | q \in Q \wedge q \notin Q_{\text{hist}}, V_{\text{jump}}^q(v_i) \neq \emptyset\}, & \text{if not empty} \\ \{q | q \in Q, q \neq \phi(v_i), V_{\text{jump}}^q(v_i) \neq \emptyset\}, & \text{otherwise} \end{cases}$$

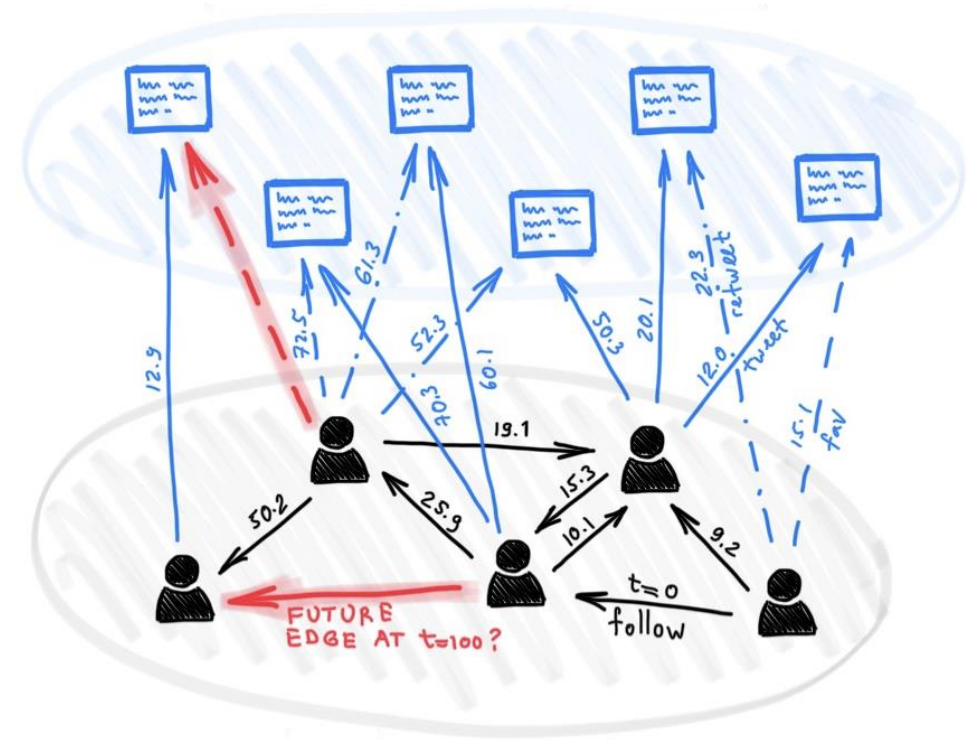


- Target domain sampled uniformly from $Q_{\text{Jump}}(v_i)$
- Larger $m \rightarrow$ higher entropy of domain distribution $\rightarrow$ more balanced walks
- Valid range: $m \in [1, |Q| - 1]$

- Previous random walk methods – DeepWalk, Node2Vec, LINE, Struc2Vec, Metapath2Vec, etc – are relied on static graph
- However, the networks in the real world are dynamic
 - Evolving over time



An illustration of user-item graph



An illustration of social graph

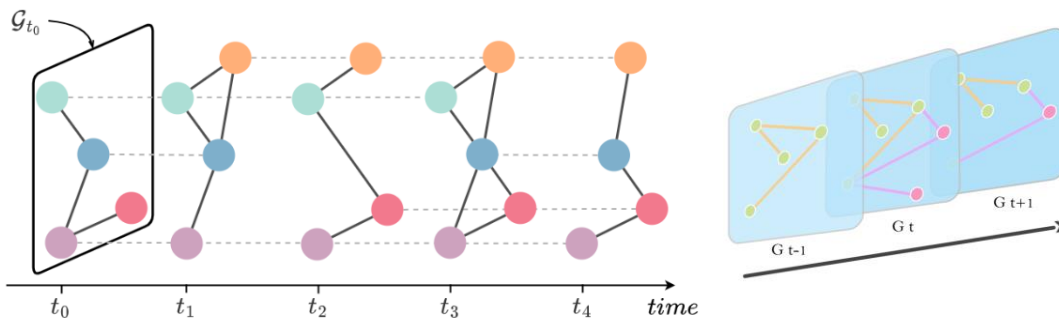
Feature	Static Graph	Dynamic Graph
Topology	Fixed	Changes over time
Nodes & edges	Constant	Nodes may join/leave; edges may form/dissolve
Attributes	Usually fixed	May evolve (weights, labels, etc.)
Representation	Single snapshot	Sequence of snapshots or event streams
Temporal reasoning	Not required	Essential for evolution, causality, temporal patterns
Algorithmic complexity	Generally simpler	Higher due to time dimension
Storage & computation	Simpler to store, visualize, compute	Must support updates, streaming, or event logs

- **Problem:** Learning dynamic node representations
- **Challenges:**
 - Time-varying graph structures: links and node can emerge and disappear; communities are changing all the time
 - Requires the node representations capture both **structural proximity** (as in static cases) and their **temporal evolution**
 - Time intervals of events are uneven
 - Causes of the change: can come from different aspects, e.g.in co-authorship network, research community & career stage perspectives
 - Requires modeling multi-faceted variations

Discrete-Time Dynamic Graphs (D-TDG)

Discrete model: Sequence of network snapshots within a given time interval

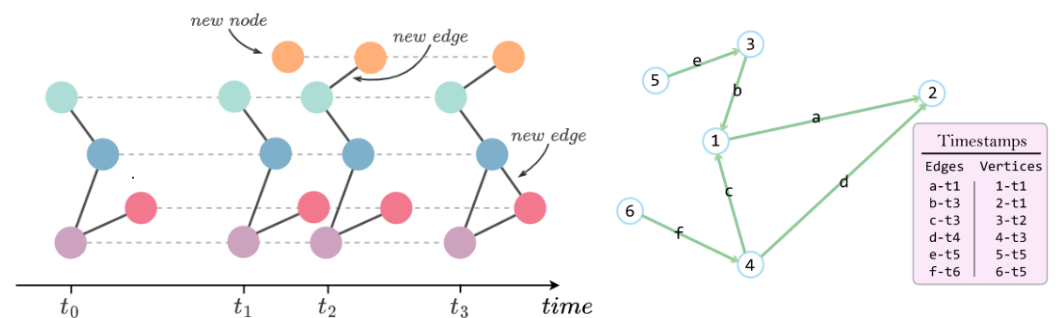
$G = \{G_1, \dots, G_T\}$, where T is the number of snapshots. Each snapshot $G_t = (V_t, E_t)$ is a static network recorded at time t



Continuous-Time Dynamic Graphs (C-TDG)

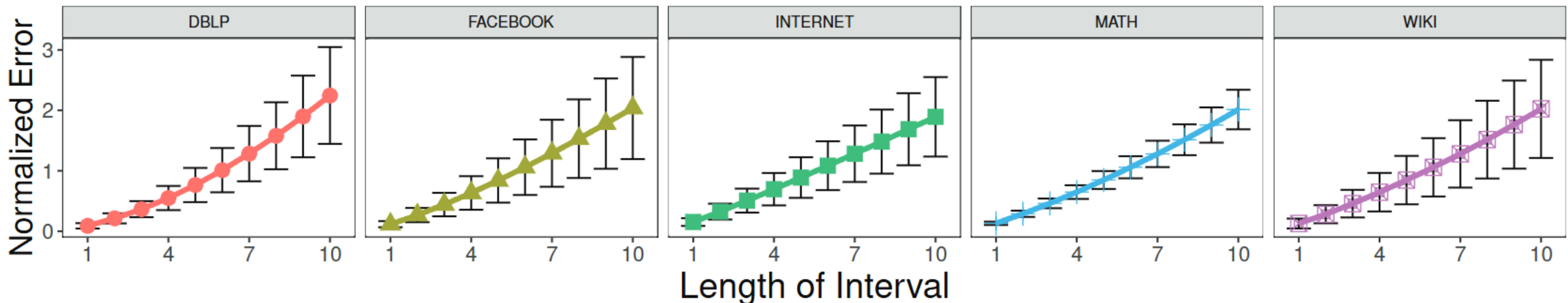
Continuous model: a network with edges and nodes annotated with timestamps

We have $G = (V_T, E_T, \mathcal{T})$ where $\mathcal{T}: V, E \rightarrow \mathbb{R}^+$ is a function that maps each edge and node to a corresponding timestamp



➤ **Motivation:** Singular Value Decomposition (SVD) on Dynamic Networks

- SVD is foundational for network tasks: link prediction, spectral analysis, parameter characterization
- Real-world networks are **dynamic** - nodes/edges appear and disappear continuously
- **Incremental SVD** avoids full recomputation, but introduces **approximation error** at each step
- Error accumulates over time → SVD must be restarted



➤ Key Notations and Definitions

- **Dynamic Network:** Represented as a sequence of adjacency matrices $A_0, A_1, \dots, A_T$ where T is the number of time slices.
- **Adjacency Matrix (A_t):** The state of the network at time t .
- **Dynamic Change (ΔA_t):** Defined as $\Delta A_t = A_t - A_{t-1}$, representing edge additions or deletions.

➤ SVD Reconstruction Loss

- The objective of Singular Value Decomposition (SVD) for networks is to minimize the reconstruction loss J :

$$J = \|S - U \cdot \Sigma \cdot V^T\|_F^2$$

Where:

- S : Symmetric similarity matrix derived from the adjacency matrix (e.g., $S = S(A)$).
- $U, V \in \mathbb{R}^{N \times k}$: Unitary matrices representing node embeddings.
- $\Sigma \in \mathbb{R}^{k \times k}$: Non-negative diagonal matrix containing the top- k singular values.
- k : Target dimensionality of the low-rank space.

➤ Minimum Model Loss

- The intrinsic minimum loss $L(S, k)$ of the SVD model (which cannot be reduced by restarting) is:

$$L(S, k) = \sum_{l=k+1}^N \lambda_l^2$$

Where:

- λ_l : Eigenvalues of S in descending order of magnitude.
- N : Total number of nodes.
- The loss is the sum of squares of the discarded eigenvalues beyond the top k .

➤ Incremental SVD Update Function $F(\cdot)$

- The function $F(\cdot)$ represents the **incremental update mechanism** that integrates new network changes without performing a full SVD restart.
- **The Update Equation**

$$[U_t, \Sigma_t, V_t] = F([U_{t-1}, \Sigma_{t-1}, V_{t-1}], S_{t-1}, \Delta S_t)$$

Where:

- $[U_{t-1}, \Sigma_{t-1}, V_{t-1}]$: The **previous SVD results** (singular vectors and values) from time slice $t - 1$
- S_{t-1} : The **similarity matrix** from the previous time slice.
- ΔS_t : The **change in similarity** at the current time slice (e.g., new edges or weight shifts).
- $[U_t, \Sigma_t, V_t]$: The **approximated results** for the current time slice.

➤ The Optimization Goal: Maximum-Error-Bounded Restart

- The goal is to minimize the total number of restarts while keeping the accumulated error below a threshold Θ :

$$\min \sum_{t=1}^T c_t \text{ subject to } G \leq \Theta$$

- $c_t \in \{0,1\}$: Binary indicator where 1 triggers an SVD restart at time t , and 0 uses incremental updates.
- G : The maximum relative error caused by incremental approximations, defined as:

$$G = \max_{1 \leq t \leq T} \frac{J(t) - L(S_t, k)}{L(S_t, k)}$$

- $J(t)$: Actual reconstruction loss at time t using the current (potentially stale) SVD results.
- $L(S_t, k)$: Theoretical minimum loss if SVD were restarted at time t .

The Framework for Error Monitoring

Challenge: Computing $L(S_t, k)$ exactly requires a full SVD same cost as a restart itself.

Efficiency via Lower Bounding

TIMERS circumvents this by using a theoretical lower bound $B(t)$:

$$L(S_t, k) \geq B(t)$$

By substituting $L(S_t, k)$ with $B(t)$, the monitoring condition becomes a sufficient condition for the error bound:

$$\frac{J(t) - B(t)}{B(t)} \leq \Theta \rightarrow \frac{J(t) - L(S_t, k)}{L(S_t, k)} \leq \Theta$$

→ Benefit: $B(t)$ is calculated in linear time relative to local dynamic changes, making real-time monitoring feasible

A Lower Bound of SVD Minimum Loss

To avoid the high cost of calculating the exact SVD minimum loss at every time step, TIMERS employs a lower bound derived from matrix perturbation theory (Stewart 1990).

The Lower Bound Theorem

If the similarity matrix S and its change ΔS are symmetric, the new minimum loss is bounded as follows:

$$L(S + \Delta S, k) \geq L(S, k) + \Delta \text{tr}^2(S + \Delta S, S) - \sum_{l=1}^k \lambda_l$$

where $\lambda_1 \geq \lambda_2 \dots \geq \lambda_k$ are the top- k eigenvalues of $\nabla_{S^2} = \mathbf{S} \cdot \Delta \mathbf{S} + \Delta \mathbf{S} \cdot \mathbf{S} + \Delta \mathbf{S} \cdot \Delta \mathbf{S}$, and

$$\Delta \text{tr}^2(\mathbf{S} + \Delta \mathbf{S}, \mathbf{S}) = \text{tr}((\mathbf{S} + \Delta \mathbf{S}) \cdot (\mathbf{S} + \Delta \mathbf{S})) - \text{tr}(\mathbf{S} \cdot \mathbf{S}).$$

Using the Bound in Monitoring

Setting the bound $B(t)$

The bound is defined by integrating the results from the last SVD restart with the accumulated structural changes:

$$B(t) = L(S_{t'}, k) + \Delta \text{tr}^2(S_t, S_{t'}) - \Lambda_k(\nabla S^2)$$

- $L(S_{t'}, k)$: The **baseline minimum loss** calculated during the most recent full SVD restart at time t' .
- $\Delta \text{tr}^2(S_t, S_{t'})$: The **change in total matrix energy** between the current time t and the restart time t' .
- $\Lambda_k(\nabla S^2)$: The **sum of the top- k eigenvalues** of the perturbation matrix ∇S^2 .

Update Logic: If the monitoring condition $\frac{J(t) - B(t)}{B(t)} > \Theta$ is met, a new SVD is triggered, and t' is updated to the current t , resetting the baseline for the bound.

Complexity Analysis

The bound $B(t)$ is highly efficient because it focuses only on local changes rather than the entire network:

Complexity:

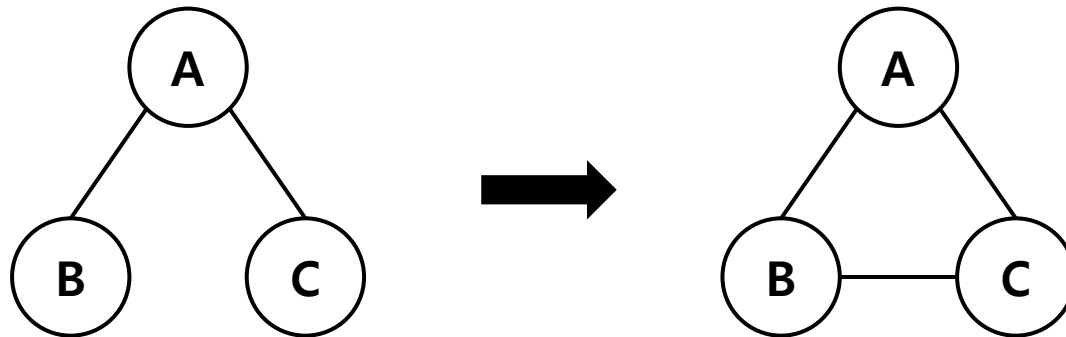
$$O(M_S + M_L k + N_L k^2)$$

- M_S : Number of changed elements in the similarity matrix.
- M_L, N_L : Number of non-zero elements/rows in ∇S^2 .
- **Linearity**: For most real-world networks (e.g., preferential attachment), the complexity is linear with respect to the number of local dynamic changes

Core Concept: Triadic Closure

The triadic closure process is a fundamental mechanism in the formation and evolution of dynamic networks. It describes how relationships develop among groups of three vertices.

- **Open Triad:** A group of three vertices where only two pairs are connected (e.g., v_i and v_j are both friends with v_k , but not with each other).
- **Closed Triad:** A group where all three vertices are connected to each other.
- **The Process:** Modeling how an open triad evolves into a closed triad over time.



Mathematical Modeling

The model assumes a vertex v_k decides whether to introduce its friends v_i and v_j based on their distance in the latent space, represented by the vector x_{ijk}^t :

$$x_{ijk}^t = w_{ik}^t \times (u_k^t - u_i^t) + w_{jk}^t \times (u_k^t - u_j^t)$$

- w_{ik}^t : Tie strength between v_i and v_k at time t .
- u_i^t : Embedding vector of vertex i at time t .

Probability of Closure

The probability that an open triad (v_i, v_j, v_k) becomes closed at time $t + 1$ is modeled as:

$$P_{tr}^t(i, j, k) = \frac{1}{1 + \exp(-\langle \theta, x_{ijk}^t \rangle)}$$

- θ : A parameter vector capturing social strategy.

Probability of Link Creation

A pair of unlinked vertices v_i and v_j often shares multiple common neighbors $B^t(i, j)$. The model assumes that a new link e_{ij} is created if at least one common friend successfully introduces them.

Probability of Link Formation at $t + 1$:

$$P_{tr+}^t(i, j) = \sum_{\alpha_{t,i,j} \neq 0} \prod_{k \in B^t(i, j)} (P_{tr}^t(i, j, k))^{\alpha_{t,i,j}^k} \times (1 - P_{tr}^t(i, j, k))^{(1 - \alpha_{t,i,j}^k)}$$

- $\alpha_{t,i,j}^k = 1$: If the open triad (v_i, v_j, v_k) develops into a closed triad.
- The sum captures all scenarios where at least one friend v_k bridges the connection.

Probability of No Link

If v_i and v_j are not influenced by any of their common friends, the open triads remain open, and no edge is formed at the next time step.

Probability of Edge Not Being Created:

$$P_{tr-}^t(i, j) = \prod_{k \in B^t(i, j)} (1 - P_{tr}^t(i, j, k))$$

- This represents the joint probability that every common neighbor v_k fails to close the triad.
- This formula allows the model to learn from "negative" instances (pairs that stay unlinked).

Loss Functions

Three components in the DynamicTriad objective:

1. Triadic closure loss:

$$\mathcal{L}_{\text{tr}}^t = - \sum_{(i,j) \in S_+^t} \log P_{\text{tr}+}^t(i,j) - \sum_{(i,j) \in S_-^t} \log P_{\text{tr}-}^t(i,j)$$

where S_+^t = links created at $t + 1$, S_-^t = links not created at $t + 1$

2. Social homophily loss:

$$L_{\text{sh}}^t = \sum_{\substack{(j,k) \in E_+^t \\ (j',k') \in E_-^t}} h(w_{jk}, [g^t(j,k) - g^t(j',k') + \xi]_+)$$

Connected pairs should be closer in embedding space than disconnected pairs

3. Temporal smoothness loss:

$$\mathcal{L}_{\text{smooth}}^t = \sum_{i=1}^N |u_i^t - u_i^{t-1}|_2^2$$

Network evolution is assumed locally smooth in the latent space

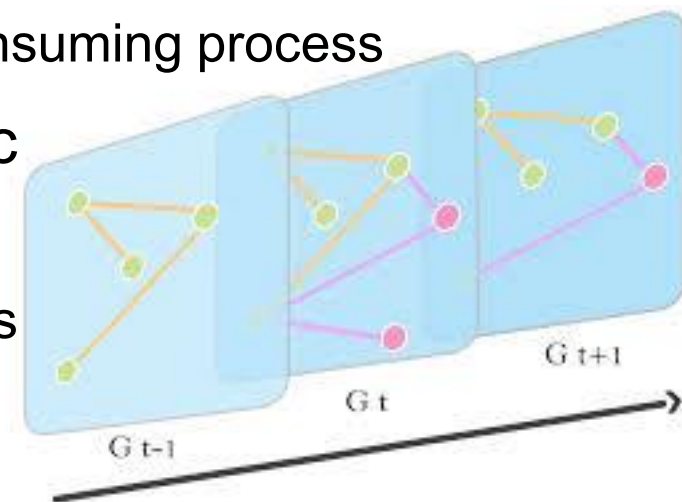
Full Objective

$$\arg \min_{\{\mathbf{u}_i^t\}, \boldsymbol{\theta}} \sum_{t=1}^T (\mathcal{L}_{\text{sh}}^t + \beta_0 \mathcal{L}_{\text{tr}}^t + \beta_1 \mathcal{L}_{\text{smooth}}^t)$$

Hyperparameter	Role
β_0	Weight of triadic closure dynamics
β_1	Weight of temporal smoothness regularization

- $\boldsymbol{\theta}$: shared social strategy vector - learned globally across all triads
- $\mathbf{u}_i^t$: the low-dimensional representation (a vector of length d) of a specific vertex i at a specific time step t .

- **Motivation:** Networks in the real world are always evolving
 - New users (new nodes) in social networks, new citations (new edges) in citation networks
 - Users may delete friends (delete edges) or some users may leave the network (delete nodes)
- The static graph embedding methods are not capable of dealing with the critical challenge involved in dynamic networks
 - **Disadvantage:** Embedding vectors for each timestamp are in different spaces
 - Leading to learn embedding vectors separately is a time-consuming process
- **Solution:** dynnode2vec method to modify the node2vec method
 - Employing the previous learned embedding vectors as initials weights for the skip-gram model

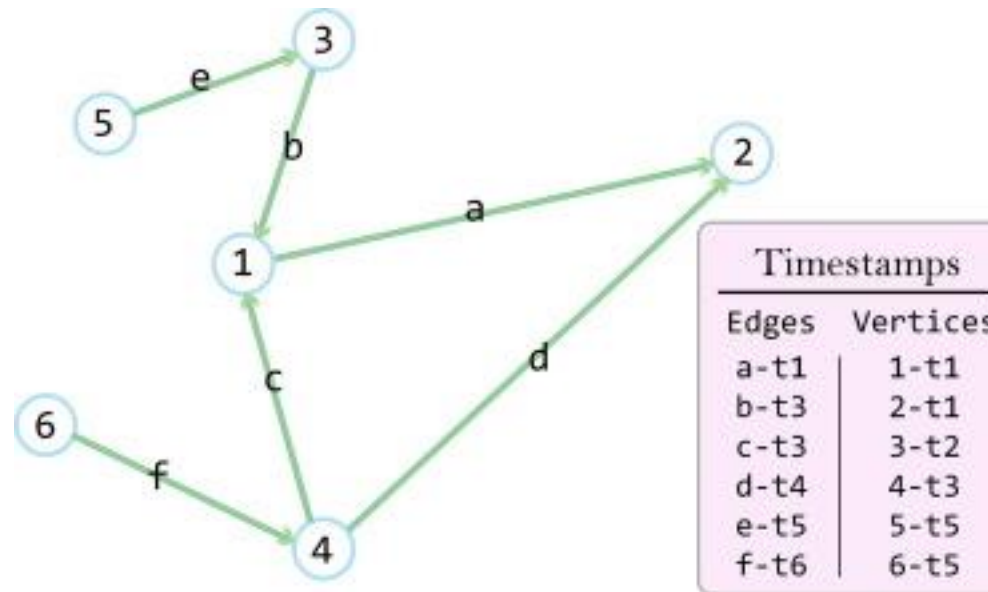


- Given a dynamic graph as a sequence $G_1, G_2, \dots, G_T$ from timestamp 1 to T
- Each graph at time t is defined as $G_t = (V_t, E_t)$
- **Evolving Random Walk Generation:**
 - Only generates random walks for the set of "evolving nodes" (ΔV_t) that have changed between consecutive timestamps t and $t+1$:

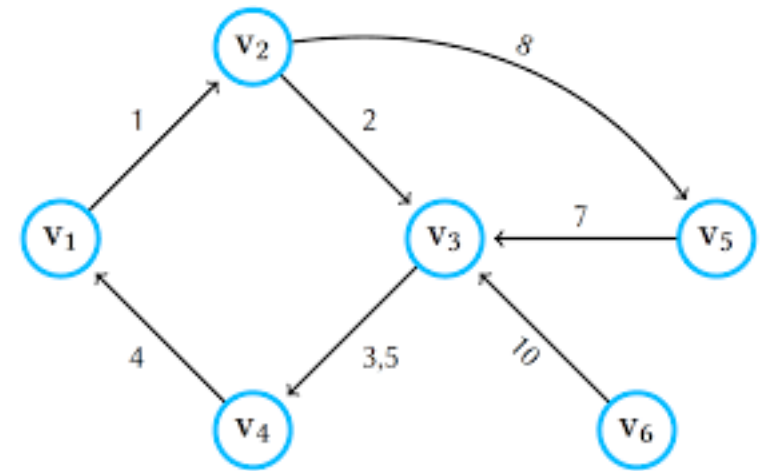
$$\Delta V_t = V_{add} \cup \{v_i \in V_t \mid \exists e_i = (v_i, v_j) \in (E_{add} \cup E_{del})\}$$

- Change: new nodes added, existing nodes deleted, or edges added/removed for existing nodes)
- **Dynamic Skip-gram Model:**
 - Initializes the skip-gram model at timestamp t with the pre-trained embedding vectors from the previous timestamp $t-1$
 - Vocabulary is updated based on the new evolving random walks, and Skip-gram t is retrained using only the new evolving random walks on the evolving node set ΔV_t

- **Motivation:** Traditional methods often treat dynamic network as a sequence of static snapshots, which loss important temporal information
- **Solution:** Treat network as a continuous-time dynamic network, capturing the exact times of interaction



- A continuous-time dynamic network is defined as $G = (V, E_T, \mathfrak{T})$
 - V : set of nodes
 - $E_T \subseteq V \times V \times \mathbb{R}^+$: set of temporal edges between vertices in V
 - $\mathfrak{T}: E \rightarrow \mathbb{R}^+$: a function that maps each edge to a corresponding timestamp
- Temporal walk:
 - A temporal walk from v_1 to v_k in G is a sequence of nodes $\langle v_1, v_2, \dots, v_k \rangle$ such that
 - $\langle v_i, v_{i+1} \rangle$ for $1 \leq i < k$ and
 - $\mathfrak{T}(v_i, v_{i+1}) \leq \mathfrak{T}(v_{i+1}, v_{i+2})$ for $1 \leq i < (k - 1)$



➤ **Temporal random walk:**

➤ The set of temporal neighbors of a node v at time t :

$$\Gamma_t(v) = \{(w, t') \mid e = (v, w, t') \in E_T \wedge \mathcal{T}(e) > t\}$$

➤ Unbiased selection: each temporal neighbor w of node v at time t is selected

$$\Pr(w) = 1/|\Gamma_t(v)|$$

➤ Biased selection: sampling the next node in a temporal walk via temporally weighted distribution based on

$$\Pr(w) = \frac{\exp[\tau(w) - \tau(v)]}{\sum_{w' \in \Gamma_t(v)} \exp[\tau(w') - \tau(v)]}$$

Exponential decay: selecting a neighbor decreases exponentially with time

$$\Pr(w) = \frac{\delta(w)}{\sum_{w' \in \Gamma_t(v)} \delta(w')}$$

Linear decay: sorts temporal neighbors in descending order time-wise.

- Temporal context windows: To handle the temporal nature of walks - a walk to run out of temporally valid edges to traverse
 - A walk must have a minimum length ω and can extend up to a maximum length L
 - The number of context windows is defined

$$\beta = \sum_{i=1}^k |\mathcal{S}_{t_i}| - \omega + 1$$

- Learning time-preserving embeddings: maximize the likelihood of observing temporal context windows given the embeddings

$$\max_f \log \Pr(W_T = \{v_{i-\omega}, \dots, v_{i+\omega}\} \setminus v_i \mid f(v_i))$$

➡ Utilizing stochastic gradient descent



네트워크 과학연구실
NETWORK SCIENCE LAB



가톨릭대학교
THE CATHOLIC UNIVERSITY OF KOREA

